

STM 32 Based Emulator Design of Permanent Magnet Synchronous Motor

PROJECT REPORT

submitted by

Anand P S CB.EN.P2EBS24024

*in partial fulfillment for the award of the degree
of*

MASTER OF TECHNOLOGY

IN

EMBEDDED SYSTEMS



AMRITA SCHOOL OF ENGINEERING, COIMBATORE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

April 2026

AMRITA VISHWA VIDYAPEETHAM

AMRITA SCHOOL OF ENGINEERING, COIMBATORE - 641112



BONAFIDE CERTIFICATE

This is to certify that the project report entitled, **STM32 Based Emulator Design of Permanent Magnet Synchronous Motor**, submitted by **Anand P S - CB.EN.P2EBS24024**, in partial fulfillment of the requirements for the award of the degree of Master of Technology in **EMBEDDED SYSTEMS**, is a bonafide record of work carried out under my guidance and supervision at Amrita School of Engineering, Coimbatore.

Dr. Supriya P

SUPERVISOR

Associate Professor

Department of Electrical

and Electronics Engineering

Amrita School of Engineering

Coimbatore - 641112

Dr. S. Balamurugan

CHAIRPERSON

Professor

Department of Electrical

and Electronics Engineering

Amrita School of Engineering

Coimbatore - 641112

This project report was evaluated by us on _____

Signature of Internal Examiner

Name

INTERNAL EXAMINER

Signature of External Examiner

Name

EXTERNAL EXAMINER

AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF ENGINEERING
COIMBATORE – 641112

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

DECLARATION

I, Mr. **Anand P S** (Register Number: **CB.EN.P2EBS24024**), hereby declare that this project report entitled **STM32 Based Emulator Design of Permanent Magnet Synchronous Motor**, is a record of the original work done by me under the guidance of **Dr. P. Supriya**, Professor, Department of Electrical and Electronics Engineering, Amrita School of Engineering, Coimbatore.

This work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or any similar award to any candidate in any university, to the best of my knowledge.

Place: Coimbatore

Signature of Student

Date:

COUNTERSIGNED

Dr. P. Supriya

Supervisor

Professor

Department of Electrical and Electronics Engineering

Amrita School of Engineering, Coimbatore – 641112

ACKNOWLEDGEMENT

I express my sincere gratitude to my guide, **Dr. P. Supriya**, Professor, Department of Electrical and Electronics Engineering, Amrita School of Engineering, for her invaluable guidance, continuous support, and encouragement throughout the course of this work.

I also take this opportunity to express my heartfelt thanks to all the faculty and staff members of the Department of Electrical and Electronics Engineering for their support and suggestions during the course of the work.

I am deeply grateful to my parents and my brother for their constant encouragement, support, and blessings.

I also thank my classmates and friends who directly or indirectly contributed to the successful completion of this work.

Above all, I express my heartfelt gratitude to the Almighty for providing me with the strength, determination, and perseverance to complete this work.

Abstract

Permanent Magnet Synchronous Motors (PMSMs) are widely employed in modern electric drives due to their high efficiency and precise control capabilities. Real-time testing of physical PMSMs can be expensive, risky, and inflexible during development. This work presents the design and implementation of a PMSM emulator based on the STM32 microcontroller platform, which replicates motor behavior through a mathematically modeled third-order transfer function. Different discretization techniques like Zero-Order Hold (ZOH), First-Order Hold (FOH), Tustin bilinear transform, and Matched Z-transform—are applied and analyzed across a range of sampling times. The emulator is verified in MATLAB for frequency and time-domain fidelity, ensuring accurate replication of PMSM speed dynamics. The best Discretization method and sampling time is used to design the digital filter whose logic is then implemented to STM 32 Development Board to mimic the dynamics of PMSM. The implemented system offers a cost-effective, safe, and scalable solution for Hardware-in-the-Loop testing, accelerating embedded controller development without requiring physical hardware.

Contents

List of figures	v
List of tables	vi
List of symbols	vii
Abbreviations	1
1 Introduction	2
1.1 Introduction	2
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Proposed Solution	3
1.5 Anticipated Outcomes	4
1.6 Thesis Organization	4
1.7 Conclusion	5
2 Literature Survey	6
2.1 Introduction	6
2.2 Literature Survey	6
2.2.1 FPGA-Based Motor Emulation and Control	6
2.2.2 Transfer Function Modeling and Mathematical Analysis	7
2.2.3 Discretization Techniques and Real-Time Implementation	7
2.2.4 Digital Filter Design and Signal Processing	7
2.2.5 Stability Analysis and Control Theory	8
2.2.6 Advanced Motor Control and Emulation Techniques	8
2.3 Research Gaps and Contributions	8
2.4 Conclusion	9
3 System Overview	10
3.1 Introduction	10
3.2 System Overview	10
3.3 Hardware Components	10
3.3.1 Hardware Setup	11
3.4 Methodology	13
3.5 Conclusion	13
4 Solution Methodology	14
4.1 Introduction	14
4.2 Methodology	14

4.2.1	Selection of Motor for Emulation	15
4.2.2	Mathematical Modelling of Transfer Function of PMSM Motor	16
4.2.3	ANSYS Design of PMSM	17
4.2.4	Discretization	18
4.3	Performance Metrics for the PMSM Emulator	19
4.3.1	Performance Metrics for the STM32-Based PMSM Emulator	19
4.3.2	Stability Criteria	21
4.4	Embedded Implementation Progress	21
4.4.1	Overview of Embedded Implementation	21
4.4.2	Development Environment and Target Hardware	22
4.4.3	System Clock Configuration	22
4.4.4	Hardware Timer-Based Sampling	22
4.4.5	ADC Interface for Input Acquisition	23
4.4.6	HAL-Based Programming Approach	24
4.4.7	Digital Filter Realization of Discrete PMSM Model	24
4.4.8	Interrupt-Driven Execution Architecture	25
4.4.9	ADC-to-Speed Mapping	25
4.4.10	UART-Based Verification	25
4.4.11	Embedded Implementation Challenges	25
4.4.12	Hardware Results using UART (PuTTY)	26
4.4.13	Conclusion	27
5	Results	29
5.1	Introduction	29
5.2	Results	29
5.2.1	Results at Sampling Time $T_s = 0.001$ s	29
5.2.2	Results at Sampling Time $T_s = 0.01$ s	31
5.2.3	Results at Sampling Time $T_s = 0.1$ s	33
5.2.4	Results at Sampling Time $T_s = 1$ s	36
5.2.5	Results at Sampling Time $T_s = 2$ s	38
5.2.6	Results at Sampling Time $T_s = 3$ s	39
5.2.7	Results at Sampling Time $T_s = 5$ s	42
5.2.8	Results at Sampling Time $T_s = 10$ s	44
5.3	Embedded Validation Results	46
5.3.1	UART-Based Validation	46
5.4	PMSM Emulator User Interface and Analysis	48
5.4.1	Overview	48
5.4.2	Continuous and Discrete Model Input	48
5.4.3	Discrete System Representation	48
5.4.4	Step Response Analysis	49
5.4.5	Frequency Response (Bode Plot)	49

5.4.6	Performance Metrics	50
5.4.7	Stability Analysis	50
5.4.8	Sampling Time Sweep	50
5.4.9	PMSM Emulator Output	51
5.4.10	Validation using Field-Oriented Control (FOC)	51
5.4.11	Conclusion	53
6	Conclusion	54
6.1	Conclusion	54
	References	55

List of Figures

3.1	System Overview	11
3.2	Experimental hardware setup of the STM32-based PMSM emulator showing the NUCLEO-F401RE board, I2C-based LCD module, and ADC input interface. . .	12
4.1	Photograph of the PMSM selected for emulation	16
4.2	Equivalent circuit of PMSM motor	16
4.3	Simulation in ANSYS.	18
4.4	Flowchart of the discretization process for PMSM transfer function	19
4.5	Real-time UART Output of PMSM Emulator displayed in PuTTY	28
5.1	Step responses of continuous and discretized systems at $T_s = 0.001$ s.	30
5.2	General trends for $T_s = 0.001$ s: step responses, error signals, Bode plots, and pole-zero map.	31
5.3	Step responses for continuous and discretized systems at $T_s = 0.01$ s.	32
5.4	Summary plots for $T_s = 0.01$ s: step responses, error signals, Bode plots, and pole-zero map.	33
5.5	Step responses at $T_s = 0.1$ s.	34
5.6	Summary plots for $T_s = 0.1$ s: step responses, error signals, Bode plots, and pole-zero map.	35
5.7	Step responses for continuous and discretized systems at $T_s = 1$ s.	36
5.8	Summary plots for $T_s = 1$ s.	37
5.9	Step responses for continuous and discretized systems at $T_s = 2$ s.	38
5.10	Performance for $T_s = 2$ s.	39
5.11	Step responses for continuous and discretized systems at $T_s = 3$ s.	40
5.12	Summary plots for $T_s = 3$ s: step responses, error signals, Bode plots, and pole-zero map.	41
5.13	Step responses for continuous and discretized systems at $T_s = 5$ s.	42
5.14	Summary plots for $T_s = 5$ s: step responses, error signals, Bode plots, and pole-zero map.	43
5.15	Step responses for continuous and discretized systems at $T_s = 10$ s.	44
5.16	Summary plots for $T_s = 10$ s: step responses, error signals, Bode plots, and pole-zero map.	45
5.17	Real-time UART outputs from STM32-based PMSM emulator.	47
5.18	Continuous vs Discrete Step Response	49
5.19	Bode Magnitude and Phase Comparison	49
5.20	Performance Metrics and TS Sweep Results	51
5.21	PMSM Emulator Output Interface	51
5.22	FOC-based PMSM control model implemented in MATLAB/Simulink	52

5.23 FOC validation of PMSM characteristics 52

List of Tables

4.1	Nameplate details of the PMSM	15
5.1	Time-domain error metrics at $T_s = 0.001$ s	30
5.2	Frequency-domain metrics at $T_s = 0.001$ s	30
5.3	Time-domain error metrics at $T_s = 0.01$ s	32
5.4	Frequency-domain metrics at $T_s = 0.01$ s	33
5.5	Time-domain error metrics at $T_s = 0.1$ s	34
5.6	Frequency-domain metrics at $T_s = 0.1$ s	35
5.7	Time-domain error metrics at $T_s = 1$ s	36
5.8	Frequency-domain metrics at $T_s = 1$ s	37
5.9	Time-domain error metrics at $T_s = 2$ s	38
5.10	Frequency-domain metrics at $T_s = 2$ s	39
5.11	Time-domain error metrics at $T_s = 3$ s	40
5.12	Frequency-domain metrics at $T_s = 3$ s	41
5.13	Time-domain error metrics at $T_s = 5$ s	42
5.14	Frequency-domain metrics at $T_s = 5$ s	43
5.15	Time-domain error metrics at $T_s = 10$ s	44
5.16	Frequency-domain metrics at $T_s = 10$ s	45

List of Symbols

F_q	–	Finite field with q elements
$\oplus$	–	Exclusive or (XOR)
ω	–	Angular Velocity
s	–	Frequency Domain
z	–	Discrete Domain
λ	–	Flux Linkage
T	–	Sampling Time
R_a	–	Armature Resistance
R_s	–	Stator Resistance
V_q	–	quadrature-axis Voltage
V_d	–	Direct- axis Voltage
P	–	Number of Pole pairs
T_l	–	Load Torque
T_e	–	Electromagnetic Torque
i_q	–	Quadrature-axis Current
L_s	–	Stator Inductance
L_a	–	Armature Inductance
K_t	–	Torque constant
K_b	–	Back EMF Constant
J_m	–	Moment of Inertia
B_m	–	Coefficient of Friction
H_∞	–	H infinity Error

List of Abbreviations

TT	–	Truth Table
ANF	–	Algebraic Normal Form
PMSM	–	Permanent Magnet Synchronous Motor
EV	–	Electric Vehicle
R&D	–	Research and Development
IoT	–	Internet of Things
HiL	–	Hardware in Loop
FPGA	–	Field Programmable Gate Array
FOC	–	Field Oriented Control
IIR	–	Infinite Impulse Response
FIR	–	Finite Impulse Response
EMF	–	Electro-Motive Force
PID	–	Proportional Integral Derivative
DAC	–	Digital to Analog Converter
GUI	–	Graphical User Interface
KVL	–	Kirchhoff's Voltage Rule
V	–	Input Voltage
DF-I	–	Direct Form I
FOH	–	First Order Hold
ZOH	–	Zero Order Hold
ECU	–	Electronic Control Unit
HEV	–	Hybrid Electric Vehicle
DC	–	Direct Current
ZN	–	Ziegler Nichols
PWM	–	Pulse Width Modulation
DSP	–	Digital Signal Processing
ABS	–	Antilock Braking System
SPMSM	–	Surface Mounted PMSM
MSE	–	Mean Squared Error
RMSE	–	Root Mean Squared Error
MAE	–	Mean Absolute Error
IAE	–	Integral Absolute Error
SISO	–	Single Input Single Output

Chapter 1

Introduction

1.1 Introduction

Permanent Magnet Synchronous Motors (PMSMs), as a part of electric drive systems, are becoming increasingly essential owing to their efficiency, compact size, and torque characteristics. The motors are being used in various electric vehicles, hybrid electric vehicles, robots, space exploration, etc. The control of Permanent Magnet Synchronous Motors is essential to obtain optimal system efficiency. Testing and validation of control algorithms implemented on Electronic Control Units (ECUs) using actual Permanent Magnet Synchronous Motors is difficult. Testing using actual Permanent Magnet Synchronous Motors is costly, time-consuming, and risky owing to high voltages and currents. Moreover, in the initial stages of development, physical motors might not be available, thereby limiting the ability to perform actual motor testing. Emulator-based solutions have been identified as a potential solution to address the constraints associated with physical motor testing. Permanent Magnet Synchronous Motor Emulators can be used to simulate Permanent Magnet Synchronous Motor electrical and mechanical behavior in real-time. The benefit is achieved by creating a safer laboratory environment, flexibility, and cost savings. The focus is on designing a Permanent Magnet Synchronous Motor Emulator that can operate in real-time. ing the STM32 microcontroller. The emulator utilizes the mathematical modeling of the PMSM, the discretization of the PMSM utilizing Bilinear (Tustin), Matched-Z, First Order Hold, and Zero Order Hold, followed by the implementation of the discretized model as an Infinite Impulse Response filter. The proposed PMSM emulator is designed with the objective of serving as an essential component of the Hardware-in-the-Loop environment. The Hardware-in-the-Loop is an industry- standard technique that is utilized for the validation and verification of the motor control strategies, reducing the risks as well as the overall cost of the tests involving the actual motors. The inclusion of the proposed PMSM emulator in the HiL environment is likely to facilitate the real-time closed-loop tests of the ECUs, utilizing the actual motor responses as well as the data logging facilities. Even though the HiL environment is beyond the scope of the current project, the design of the proposed PMSM emulator is robust.

In conclusion, the proposed PMSM emulator, which is built around the STM32 microcontroller, is likely to fill the gap between the simulation environment and the actual hardware tests. Therefore, the proposed PMSM emulator is likely to contribute significantly towards the development of the motor control system.

1.2 Problem Statement

Developing reliable ECU testing platforms for automotive applications faces several challenges due to expe

ECU Validation Challenge: It is risky and expensive to test the ECUs using real PMSMs, as the PMSMs may get damaged during the test because of the high voltage or high current.

Simulation Limitation: Online simulation cannot accurately represent the real-time behavior of the interaction between the ECU and the PMSM.

Need for Real-time Emulation: It is not possible to get the desired hardware-level interaction using software modeling.

Safe & Cost-effective Alternative: It is possible to get the desired PMSM emulation using a microcontroller-based PMSM emulator, as it would be safe, flexible, and inexpensive.

All these statements clearly indicate the importance of using an STM32-based PMSM emulator for the purpose.

1.3 Objectives

Develop STM32-based Emulator Design and implement a microcontroller-based system that replicates the behavior of a PMSM. Mathematically derive the transfer function correlation angular speed ω and the input voltage V . Utilize Bilinear (Tustin), Matched Z, First Order Hold, and Zero Order Hold discretization techniques to transform different sampling times into a discretized form of the transfer function. Implement the best discretized form of the PMSM transfer function as an IIR filter using the Direct Form I method. Simulation Testing Test the developed filters/models using MATLAB/Simulink. Execute the best-performing models on an STM32 microcontroller for hardware implementation in the form of emulators.

1.4 Proposed Solution

1. In order to overcome the hurdles associated with testing direct PMSM hardware, this study proposes a real-time PMSM emulator using a microcontroller, specifically the STM32 platform. This emulator accurately emulates the PMSM motor in terms of both electrical and mechanical characteristics, thus providing a safer and cheaper way of testing ECU hardware.
2. Starting with the continuous time PMSM, we derive equations for it, then convert them into discrete time equations using various methods such as ZOH, FOH, Bilinear, and Matched Z methods for different sampling intervals. This way, we can select the appropriate sampling method for the real-time application.

3. We then implement these discrete time equations as Infinite Impulse Response (IIR) digital filters in the form of Direct Form I, which is appropriate for implementation on the STM32 microcontroller, given its low processing capabilities.
4. Finally, we validate the proposed PMSM emulator using MATLAB/Simulink, then implement it on actual hardware, thus linking the world of offline simulations with actual hardware testing.
5. This proposed PMSM emulator, being accurate, efficient, and cheaper, will greatly speed up ECU development and testing, reducing the need for expensive PMSM or HiL systems.

1.5 Anticipated Outcomes

The proposed STM32-based PMSM emulator is expected to achieve the following outcomes:

1. **Safe ECU Testing:** Provide a reliable platform for validating automotive ECUs without the risks of high voltages and currents from real PMSMs.
2. **Cost-effective Alternative:** Reduce dependency on expensive Hardware-in-the-Loop (HiL) systems by offering a low-cost microcontroller-based solution.
3. **Accurate Real-time Emulation:** Achieve faithful replication of PMSM dynamics using mathematical modeling, discretization, and digital filter realization.
4. **Performance Optimization:** Identify the best discretization method and sampling time to achieve stable and low-latency real-time operation.
5. **Research Contribution:** Close the gap between online simulation and physical motor testing by developing a feasible hardware emulator.
6. **Scalability:** Develop a scalable framework that can be extended to emulate other motors and control techniques in the future.

1.6 Thesis Organization

This thesis is organized into five main chapters as follows:

1. **Chapter 1: Introduction** – In this chapter, the background, motivation, problem statement, and objectives of the project will be presented. The importance of PMSM emulation in ECU validation will be emphasized, as well as the scope and relevance of the proposed work.
2. **Chapter 2: Literature Review** – In this chapter, a thorough review of the literature on PMSM modeling, discretization schemes, and motor emulation will be presented.

3. **Chapter 3: System Overview** -In this section, the architecture of the proposed PMSM emulator will be presented, including the mathematical model, derivation of the transfer function, discretization schemes, and IIR filter.
4. **Chapter 4: Methodology** - This section will describe the steps involved in the proposed work.
5. **Chapter 5: Conclusion** – In this concluding chapter, the benefits of the proposed emulator, as well as the potential contributions of the proposed work, will be presented.

1.7 Conclusion

This chapter will discuss the idea of Permanent Magnet Synchronous Motor (PMSM) emulation and the rationale for choosing STM32 to implement this. It will outline the objectives of the project and emphasize the importance of safe and real-time tests via emulators. It will give a quick glimpse of the structure of the rest of the report, leading to the detailed discussions of the succeeding chapters.

Chapter 2

Literature Survey

2.1 Introduction

Permanent Magnet Synchronous Motor (PMSM) emulators' design and validation have received considerable research attention owing to their significance in electric drives, embedded applications, and Hardware-in-the-Loop (HiL) simulations. This comprehensive review aims to cover a wide variety of research contributions related to FPGA implementation, motor modeling, discretization methods, digital filter design, and control strategies that are of interest for Permanent Magnet Synchronous Motor emulators.

2.2 Literature Survey

2.2.1 FPGA-Based Motor Emulation and Control

FPGA platforms have emerged as preferred solutions for real-time motor emulation due to their parallel processing capabilities and reconfigurability. Tashakori et al. [1] demonstrated a digital PWM controller for BLDC motors based on FPGA for BLDC motors, that focuses on voltage source inverter control for average torque regulation. Speed control methods for controlling DC motors using FPGA platforms were explored by Genc and Hatas [22].

Rama and Ananthan [2] developed FPGA-based DC shunt motor emulator, comparing direct form I and direct form II arithmetic and challenges. Namboothiripad [23] analyzed the impact of discretization techniques on real-time DC servo motor simulation using FPGA, providing critical insights into sampling-related performance considerations. Rosa et al. [3] presented hardware DC motor emulators emphasizing cost-effectiveness for educational and prototyping applications.

FPGA technology has been recognized as the go-to technology for motor emulation in real-time owing to the parallel processing capabilities of these technologies. Tashakori et al. presented a digital PWM controller for BLDC motor control using the FPGA technology, demonstrating voltage source inverter control for average torque regulation, while at the same time identifying the complexity of the control algorithm. Genc and Hatas presented a study on speed control methods for DC motor control on FPGA technology, thus expanding the motor control space. Rama and Ananthan developed detailed DC motor emulators on FPGA technology, focusing on various filter configurations and addressing the complexities of fixed-point arithmetic. Namboothiripad presented a study on the discretization methods and their effects on the simulation of DC servo motor in real-time on FPGA technology, thus providing crucial information on the capabilities of the technology. More advanced studies on motor emulation on FPGA technology include the work by Esparza on induction motor emulation,

while another study by Sutikno presented detailed work on developing high-precision estimators for torque and flux in direct torque control technology.

2.2.2 Transfer Function Modeling and Mathematical Analysis

The use of mathematical modeling is vital in the realistic motor emulation. In a study done by Ba and colleagues [7], generalized per-phase equivalent circuit modeling of PMSMs was proposed with the aim of modeling predictable core losses. In another study done by Dhaouadi and colleagues [11], an extended Kalman filter was proposed for state estimation in PMSM drives, which is a common practice in modern observer-based control approaches.

Some of the classic control theory concepts are evident in a number of landmark papers. In a study done by Horowitz and Shaked [12], the benefits of transfer function modeling over state variable modeling were pointed out, especially in the case of linear time-invariant feedback systems. In another study done by Brockett [14], a deeper understanding of the role of poles and zeros in state variable modeling is provided. Additionally, Snelgrove and Sedra [13] proposed active filter synthesis using intermediate transfer functions.

Further developments in system identification were presented by Stoica and Jansson [15], where multiple-input multiple-output (MIMO) systems were analyzed using both state-space and transfer function formulations.

2.2.3 Discretization Techniques and Real-Time Implementation

The accuracy of discretization is the key stone on which the faithfulness of a real-time emulator depends. [4] emphasized the importance of sampling time for the Power Hardware-in-the-Loop of the PMSM emulator and provided valuable guidelines to achieve accuracy in this field. Researchers have conducted extensive studies on the comparison of discretization techniques like the Backward Euler method, Trapezoidal Rule method, First Order Hold method, and the Tustin Transformation method to assess the accuracy of the discretization techniques. Yadav and Gupta [20] proposed the development of fractional-order differentiators and integrators using direct discretization techniques. They explored the theory of continued fraction expansion for the rational approximation of the developed techniques. They emphasized the discretization techniques like the optimized four-segment rule proposed by Al-Alaoui and Simpson's 1/3 rule.

2.2.4 Digital Filter Design and Signal Processing

The development of powerful digital filters provides the foundation for high-fidelity motor emulation. Beylkin et al.'s study [8] focused on addressing the issue of creating highly precise digital filters, such as IIR and FIR, to create optimal approximation techniques for embedded systems. Getu's study [10] focused on digital IIR filter development through bilinear transformation in MATLAB, emphasizing various aspects of digital filter implementation. Jain et al.'s study [9] focused on assessing various reduced-order digital IIR filter designs for signal denoising, emphasizing various aspects of efficiency in digital filter development for embedded

systems, such as motor control systems. Telba's study [25] focused on comparing various adaptive filter architectures in Simulink, emphasizing various aspects of digital filter development for different applications. Swain et al.'s study [18] focused on modeling inverse fractional-order band-pass and band-stop filters with fractional elements, enhancing digital filter development for various applications, such as motor systems. Morales and Agili's study [19] focused on analyzing S-parameter data using DSP techniques, showing significant advancements in frequency analysis techniques for various applications, such as characterizing motor systems.

2.2.5 Stability Analysis and Control Theory

System stability analysis is a guarantee for the reliable operation of emulators in different working states. In terms of power electronics dominant drive systems, Chen et al. in [21] provide a survey on small-signal stability criteria for PMSM emulators, pointing out eigenvalue-based analysis as a key aspect for ensuring PMSM emulator stability. Rosu in [17] presented a novel approach for implementing recursive error division Kalman filters for sophisticated adaptive filtering in motor drive applications. Spitsyn et al. in [16] presented mathematical models for linear DC motors.

2.2.6 Advanced Motor Control and Emulation Techniques

Recent advances in the field of motor emulation technology demonstrate advanced control philosophies and methods of implementation. Modern methods utilize a variety of discretization methods, adaptive filters, and robust stability tests to provide precise motor emulation over a broad range of operation.

Power Hardware-in-the-Loop (PHIL) simulation has been developed to provide a framework for the testing of various motor control algorithms without the need for the actual motor. These advancements cater to the need for precise motor emulation solutions.

2.3 Research Gaps and Contributions

Despite extensive research progress, several gaps motivate the present work:

1. There has been a lack of benchmarking of discretization methods used for PMSM emulation at various sampling rates.
2. Cost-efficient implementation using the microcontroller has not been extensively investigated compared to the implementation using FPGAs.
3. Mathematical modeling, discretization, and filtering have rarely been combined into a single system.
4. There has been a lack of validation, including both time- and frequency-domain testing, as well as hardware-in-loop testing.

2.4 Conclusion

The gaps in current literature will be filled by this dissertation, which compares and evaluates some discretization techniques, i.e., ZOH, FOH, Tustin, and Matched-Z, for emulating a PMSM using STM32 microcontrollers. The dissertation will provide detailed error analyses in the s-domain and z-domain, step response verification, and stability tests under different sampling rate conditions. The end result will be accurate and scalable emulators for PMSMs, which can be used in motor control applications.

The work will provide a balance between theoretical foundations and practical aspects, thus advancing the state of the art in cost-effective and accurate motor emulations.

Chapter 3

System Overview

3.1 Introduction

This chapter explains the proposed system architecture and the design philosophy that is followed in developing a real-time emulator for a Permanent Magnet Synchronous Motor (PMSM), which is controlled by an STM32-based controller. The emulator is designed in such a manner that it closely emulates a real-life Permanent Magnet Synchronous Motor. The emulator is particularly beneficial in research and embedded design environments where access to high-power motors is not possible and the motor is to be tested under supercritical conditions, such as in Antilock Braking Systems (ABS) in cars.

3.2 System Overview

The emulator operates on the basis of the STM32F401RE microcontroller, which is the computation core. The system emulates the performance of a PMSM through the execution of a pre-programmed logic that generates motor outputs such as angular speed. The outputs are computed on the basis of input reference signals and supplied either from DAC outputs or communication interfaces.

The equipment consists of the following:

1. An input voltage provided via potentiometer or GUI.
2. Real-time computation by using STM32 firmware.
3. Output signal Angular speed supplied on DAC channels.

The emulator interacts with other embedded systems or controllers to allow Hardware-in-the-Loop (HiL) simulations so that motor control logic may be extensively tested without having to use actual motors. The system overview is shown in Figure 3.1.

3.3 Hardware Components

The most critical pieces of hardware in the emulator design are:

1. **STM32 Discovery Board:** Is used as the emulation core processor.
2. **Potentiometer:** Employed to provide input information (Voltage simulation) to the emulator.
3. **I2C Module:** Used for LCD Display for showing the angular speed.

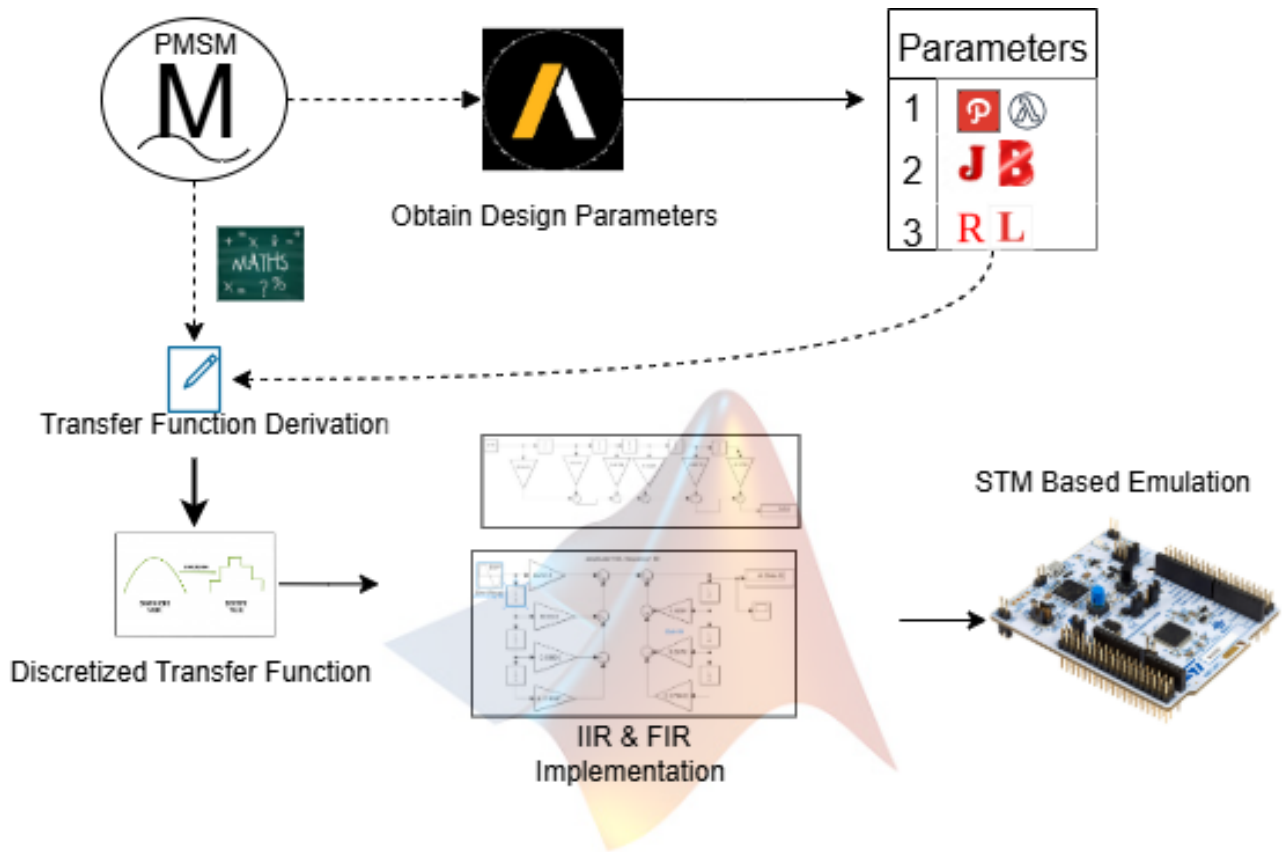


Figure 3.1: System Overview

4. **Power Supply and Debug Interfaces:** Required to power and program the board.

3.3.1 Hardware Setup

The experimental validation of the STM32-based PMSM emulator was carried out using the NUCLEO-F401RE development board. The setup includes:

- STM32F401RE NUCLEO board
- 16×2 character LCD interfaced via I2C
- Potentiometer connected to ADC input (PA0)
- UART for serial debugging (115200 baud)

The complete hardware setup used for embedded validation is shown in Fig. 3.2.

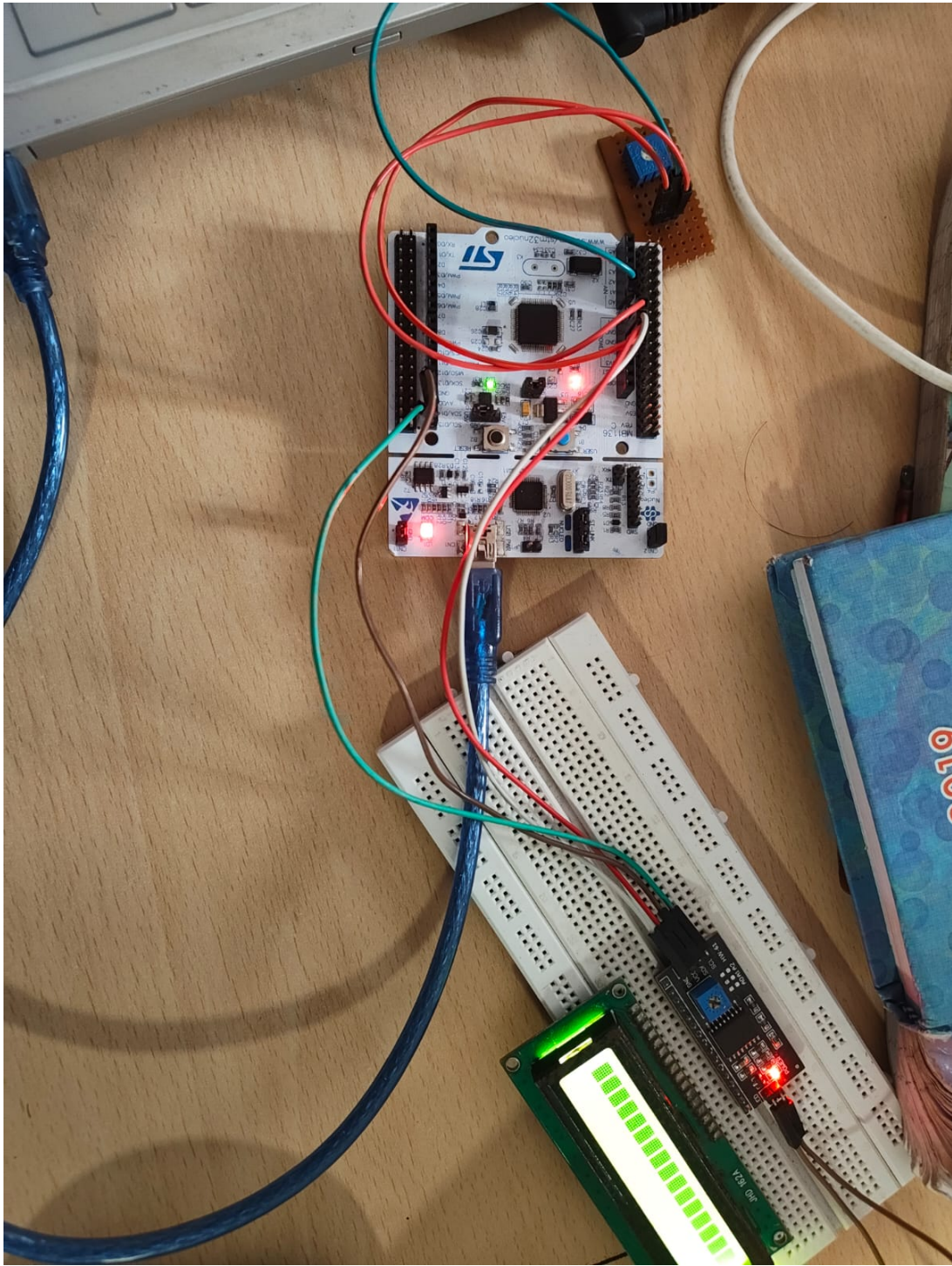


Figure 3.2: Experimental hardware setup of the STM32-based PMSM emulator showing the NUCLEO-F401RE board, I2C-based LCD module, and ADC input interface.

The STM32 processes the discretized PMSM model within a timer interrupt. The potentiometer provides an analog voltage from 0 to approximately 3.3 volts, which the ADC converts to a digital signal with 12 bits. The processed output is displayed on the LCD and simultaneously transmitted through UART for debugging and verification.

3.4 Methodology

The project follows a systematic approach to implement the PMSM emulator:

1. **Selection of Motor for Emulation:** A PMSM Motor of interest is chosen.
2. **ANSYS Design:** The PMSM model available in ANSYS is used to extract the detailed **electrical, mechanical, and magnetic parameters** essential for accurate simulation.
3. **Mathematical Modeling:** Using the motor parameters, the transfer function $\frac{\omega(s)}{V(s)}$ is derived based on the electrical and mechanical equations of PMSM.
4. **Discretization:** The continuous-time transfer function is discretized using different techniques and sampling time using MATLAB to convert it to obtain $(\frac{\omega(n)}{V(n)})$ z domain equation.
5. **Implementation in Simulink:** The best discretized transfer function model is realized as an **IIR filter** in MATLAB Simulink environment.
6. **Signal Computation:** $\omega(n)$ is computed as the product of the input voltage signal $V(n)$ and the stable discrete filter response.
7. **Microcontroller Deployment:** The final discrete equation for $\omega(n)$ is implemented in the **STM32** microcontroller to emulate real-time PMSM behavior.

3.5 Conclusion

This chapter described the architecture and methodological steps involved in designing the STM32-based PMSM emulator. It outlined the hardware setup, implementation stages, and intended functionalities of the emulator system. The next chapter will focus on the detailed mathematical modeling of the PMSM and its discrete-time implementation.

Chapter 4

Solution Methodology

4.1 Introduction

Choosing a target PMSM is the first step in the study. The basic electrical and mechanical equations of the PMSM are used to derive the mathematical transfer function that links motor speed to input voltage using the extracted parameters. ANSYS is used to model the PMSM in order to capture realistic motor dynamics. The z-domain representation of the continuous-time transfer function is then obtained by discretizing it in MATLAB using a variety of methods. To verify performance through simulation, MATLAB Simulink uses the best discretized model as an IIR filter. The input voltage multiplied by the stable filter output is the discrete motor response.

4.2 Methodology

The methodology for this work :

1. **Selection of Motor for Emulation:** A target PMSM is selected along with its name plate details to design in ANSYS Software.
2. **Mathematical Modeling:** Using the motor parameters, the transfer function $\frac{\omega(s)}{V(s)}$ is derived based on the electrical and mechanical equations of PMSM.
3. **Design in ANSYS:** The PMSM is designed in ANSYS to extract detailed **electrical, mechanical, and magnetic parameters** essential for accurate simulation.
4. **Discretization:** The continuous-time transfer function is discretized using different discretization techniques and sampling time using MATLAB to convert it to obtain $(\frac{\omega(n)}{V(n)})$ z domain equation.
5. **Implementation in Simulink:** The best discretized transfer function model is realized as an **IIR filter** in MATLAB Simulink environment.
6. **Signal Computation:** $\omega(n)$ is computed as the product of the input voltage signal $V(n)$ and the stable discrete filter response.
7. **Microcontroller Deployment:** The final discrete equation for $\omega(n)$ is implemented in the **STM32** microcontroller to emulate real-time PMSM behavior.

4.2.1 Selection of Motor for Emulation

The first step in the emulation process is the selection of a suitable Permanent Magnet Synchronous Motor (PMSM) whose behavior needs to be reproduced in real time. The motor parameters are essential in developing an accurate mathematical model, since they directly determine the electrical and mechanical equations used for transfer function derivation. For this work, a laboratory-rated PMSM was chosen as the target machine for emulation. The basic nameplate ratings of the motor are summarized in Table 4.1.

Parameter	Value
Voltage	166 V (max)
Current	3.69 A (rms)
Speed	4600 RPM
Output	2 HP

Table 4.1: Nameplate details of the PMSM

These parameters serve as the foundation for mathematical modeling, discretization, and subsequent microcontroller-based implementation. The original motor selected for this work is shown in Figure 4.1, which acts as the reference system for validating the emulation process.

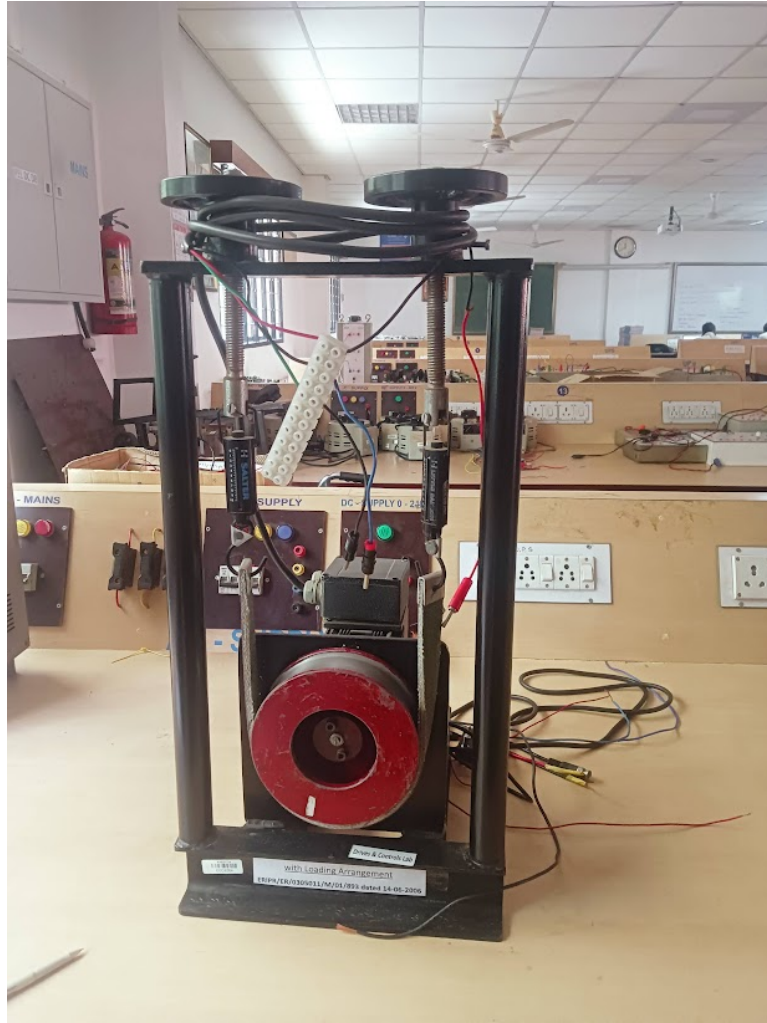


Figure 4.1: Photograph of the PMSM selected for emulation

4.2.2 Mathematical Modelling of Transfer Function of PMSM Motor

The following assumptions are considered for the derivation: Surface-mounted PMSM is used, hence $L_d = L_q = L_s$. Field-Oriented Control (FOC) strategy is assumed, with $I_d = 0$.

The equivalent circuit of the PMSM motor is shown in Fig. 4.2.

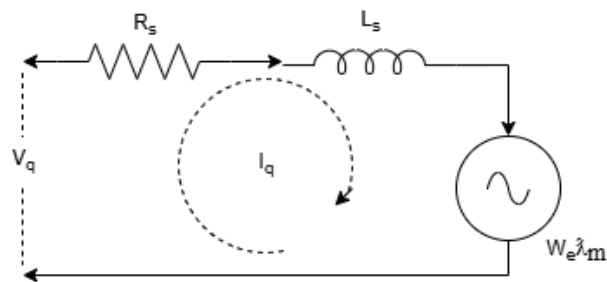


Figure 4.2: Equivalent circuit of PMSM motor

Electrical Equation by KVL: is given by (4.1):

$$V_q = R_s i_q + L_s \frac{di_q}{dt} + \omega_e \lambda_m \quad (4.1)$$

Mechanical Equations: is given by (4.2) and (4.3):

$$T_e = \frac{3}{2} P \lambda_m i_q \quad (4.2)$$

$$T_e - T_L = J \frac{d\omega}{dt} + B_m \omega \quad (T_L = 0 \text{ assumed}) \quad (4.3)$$

Taking laplace of (4.3) to obtain (4.4)

$$T_L(s) = Js\omega(s) + B_m\omega(s) \quad (4.4)$$

Taking Laplace transform of (4.1) we get (4.5) :

$$V_q(s) = R_s I_q(s) + L_s s I_q(s) + \lambda_m \omega(s) \quad (4.5)$$

Comparing equations (4.2) & (4.4)

Hence,

$$I_q(s) = \frac{(Js + B_m)\omega(s)}{\frac{3}{2}P\lambda_m} \quad (4.6)$$

Substituting (4.6) into (4.5):

$$V_q(s) = \left[\frac{(Js + B_m)}{\frac{3}{2}P\lambda_m} \omega(s) \right] (R_s + L_s s) + \lambda_m \omega(s) \quad (4.7)$$

Simplifying:

$$V_q(s) = \left[(Js + B_m)(R_s + L_s s) \cdot \frac{1}{\frac{3}{2}P\lambda_m} + \lambda_m \right] \omega(s) \quad (4.8)$$

Therefore, the transfer function is:

$$\frac{\omega(s)}{V_q(s)} = \frac{\frac{3}{2}P\lambda_m}{(Js + B_m)(R_s + L_s s) + \frac{3}{2}P^2\lambda_m^2} \quad (4.9)$$

4.2.3 ANSYS Design of PMSM

To support the analytical modelling and validate the derived transfer function, a Surface-mounted Permanent Magnet Synchronous Motor (SPMSM) was designed and analyzed using ANSYS Motor-CAD. The machine under study is a four-pole, three-phase motor with a rated power of 0.55 kW, a rated voltage of 220 V, and a rated speed of 1800 rpm at 60 Hz. The design was carried out for motor operation with constant power load, at an operating temperature of 75 °C.

Stator Design: The stator consists of 24 slots with double-layer distributed windings in a

star-connected three-phase configuration. The winding factor was obtained as 0.933, with an effective slot fill factor of about 71%. From the simulation, the stator resistance is $2.16\ \Omega$, and the stator leakage inductance is 3.7 mH. These parameters directly form the electrical dynamics of the SPMSM transfer function.

Rotor and Magnet Design: The rotor is of surface-mounted permanent magnet type, with four poles. High-energy permanent magnets are mounted on the rotor surface, giving a residual flux density of 0.96 T. The magnet flux linkage (λ_m) obtained from the simulation corresponds to the excitation term in the electrical equation of the transfer function.

Performance Results: The designed SPMSM develops a rated torque of 6.92 N m at 4600 rpm, delivering an output power close to the rated 0.55 kW. The copper loss was found to be approximately 52 W, while the iron-core loss was about 8.6 W, resulting in a total loss of 66.9 W. The efficiency of the machine is 89 %. The fundamental RMS back-EMF was observed as 71.2 V, which validates the constant terms used in the electrical dynamics.

The ANSYS design model of the SPMSM is shown in Fig. 4.3.

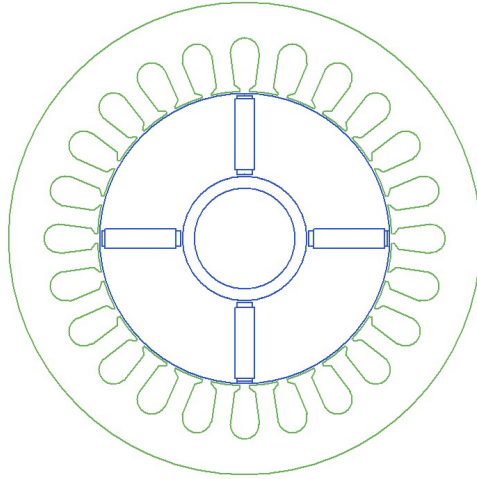


Figure 4.3: Simulation in ANSYS.

4.2.4 Discretization

The discretization is essential in the design of a digital control system. It transforms the transfer function into a discretized form that is acceptable by the microcontroller or DSP. The discretization affects the performance of the control system as well as the stability maintained by the system. The discretization techniques include zero-order hold, first-order hold, bilinear transformation, and matched pole zero transformation. These techniques produce errors depending on the sampling interval T_s .

In a practical control system, like a PMSM emulator using STM32, discretization is essential because it allows the control system to handle discretized signals instead of continuous signals.

The choice of T_s is a trade-off because a smaller T_s increases computation power but increases power consumption.

The zero-order hold is widely used when connected to a digital-to-analog converter because it is a close approximation to a true zero-order hold on the input signal. The first-order hold is a better approximation because it considers linear changes in signals. The linear changes are significant when signals change slowly. The Matched Pole-Zero method provides exact pole and zero mapping, preserving system dynamics at specified frequencies, thereby enhancing accuracy for frequency-critical applications.

A MATLAB-based discretization code (see Figure 4.4) illustrates the procedure wise implementation of these methods and forms the basis for comparative performance analysis.

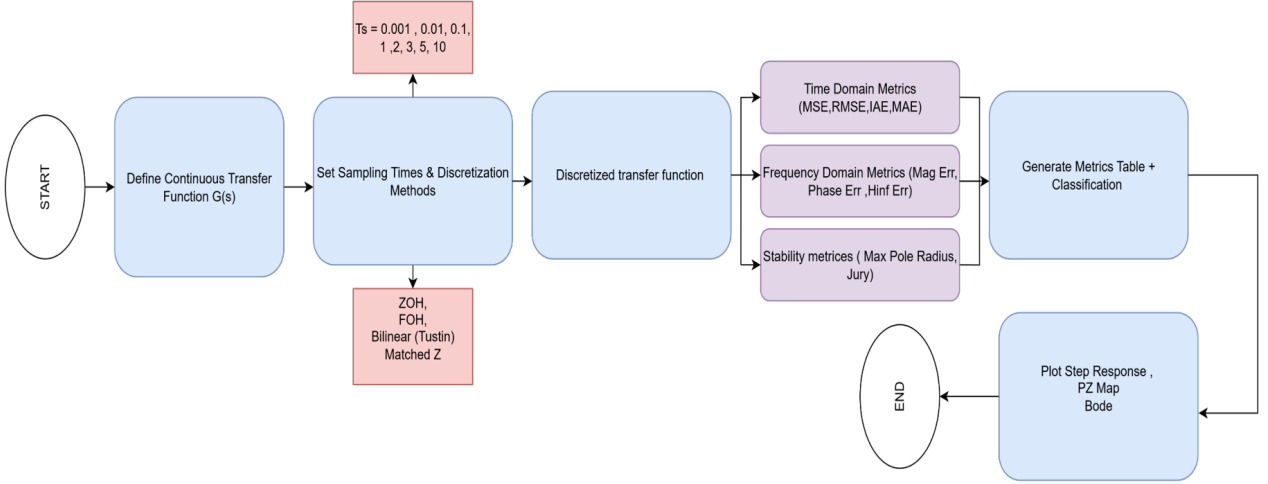


Figure 4.4: Flowchart of the discretization process for PMSM transfer function

4.3 Performance Metrics for the PMSM Emulator

For evaluating the accuracy of the STM32-based Permanent Magnet Synchronous Motor (PMSM) emulator, various time-domain and frequency-domain metrics are employed to quantify deviations from the continuous-time reference model.

4.3.1 Performance Metrics for the STM32-Based PMSM Emulator

To evaluate the accuracy of the STM32-based Permanent Magnet Synchronous Motor (PMSM) emulator, both time-domain and frequency-domain performance metrics are employed. These metrics quantify the deviation between the emulator's discrete-time output and the continuous-time reference model.

In the following, $y_{\text{ref}}[k]$ denotes the reference output of the continuous-time PMSM system at discrete sample index k , obtained via high-fidelity simulation or experimental measurement. The variable $y_{\text{model}}[k]$ represents the corresponding output of the discrete-time PMSM emulator implemented on the STM32 platform. The continuous-time frequency response at frequency

ν_i is $G_{\text{ref}}(j\nu_i)$, whereas $G_{\text{model}}(e^{j\nu_i T_s})$ is the frequency response of the emulator at the same frequency, with sampling period T_s . The total number of samples is N .

Time-Domain Metrics

1. **Mean Squared Error (MSE):** The MSE measures the average squared deviation between emulator and reference outputs, penalizing larger errors more heavily, and is defined in (4.10)

$$\text{MSE} = \frac{1}{N} \sum_{k=1}^N (y_{\text{ref}}[k] - y_{\text{model}}[k])^2 \quad (4.10)$$

2. **Root Mean Squared Error (RMSE):** The RMSE is the square root of the MSE, restoring the error to the physical units of the system output and is given by (4.11)

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N (y_{\text{ref}}[k] - y_{\text{model}}[k])^2} \quad (4.11)$$

3. **Mean Absolute Error (MAE):** The MAE computes the average of the absolute differences, providing robustness to outliers, and is defined as (4.12)

$$\text{MAE} = \frac{1}{N} \sum_{k=1}^N |y_{\text{ref}}[k] - y_{\text{model}}[k]| \quad (4.12)$$

4. **Integral of Absolute Error (IAE):** The IAE accumulates the absolute error over the total time window NT_s , representing long-term deviation, as (4.13)

$$\text{IAE} = T_s \sum_{k=1}^N |y_{\text{ref}}[k] - y_{\text{model}}[k]| \quad (4.13)$$

Frequency-Domain Metrics

1. **Magnitude Error:** The magnitude error at discrete frequency ν_i quantifies deviation in amplitude response and is defined as (4.14)

$$\text{MagError}(\nu_i) = \left| 20 \log_{10} \frac{|G_{\text{ref}}(j\nu_i)|}{|G_{\text{model}}(e^{j\nu_i T_s})|} \right| \quad (4.14)$$

2. **Phase Error:** The phase error measures the difference in phase angle at frequency ν_i , given by (4.15)

$$\text{PhaseError}(\nu_i) = \left| \angle G_{\text{ref}}(j\nu_i) - \angle G_{\text{model}}(e^{j\nu_i T_s}) \right| \quad (4.15)$$

3. **H_∞ Norm:** The H_∞ norm defines the maximum gain across all frequencies, serving as

a robustness metric given by (4.16) :

$$\|G\|_{\infty} = \sup_{\nu} |G(j\nu)| \quad (4.16)$$

These performance indices provides a rigorous framework to quantify and analyze the emulator performance for HIL validation.

4.3.2 Stability Criteria

The stability of discrete time systems is ensured if the following two criteria are satisfied:

1. **Maximum Pole Radius:** All the poles of the discrete transfer function must be strictly inside the unit circle in the z-plane. In other words, each of the poles z_i must satisfy the condition $|z_i| < 1$. This ensures that the output is bounded if the input is bounded.
2. **Jury's Stability Test:** This is based on a tabular algebraic method. Let the characteristic polynomial be

$$P(z) = a_0 z^n + a_1 z^{n-1} + \dots + a_n.$$

Then, the system is stable if

$$P(1) > 0, \quad (-1)^n P(-1) > 0,$$

and all the principal minors of the Jury array are positive. If all these conditions are satisfied, then all the poles are guaranteed to lie inside the unit circle without explicitly calculating them. These two criteria provide a quantitative measure of the performance of each of the discretization methods in preserving the properties of the original system and are related to the practical stability of the digital system. All results and interpretations in the report are based upon these rigorous performance and stability criteria.

4.4 Embedded Implementation Progress

This section describes how to follow the procedure for building a preliminary real-time PMSM emulator framework using an STM32-based microcontroller. The procedure involves building a deterministic execution environment for real-time execution of discretized motor models.

4.4.1 Overview of Embedded Implementation

The following section provides an overview of how this project was developed in steps. The objective is quite clear: to create an embedded system that can run system equations in discretized form with a constant sampling time. The development was guided by the following principles:

1. Deterministic sampling using a hardware timer.

2. Real-time analog input acquisition using ADC.
3. Modular peripheral configuration using STM32CubeIDE.
4. Hardware abstraction using STM32 HAL libraries.
5. Real-time output visualization using an LCD interface.

This implementation serves as the foundational stage for integrating the complete PMSM discrete-time model.

4.4.2 Development Environment and Target Hardware

The motor emulator is developed using STM32CubeIDE. STM32CubeIDE is an Integrated Development Environment that allows users to set up their peripherals, produce code, compile, and debug their code in one go.

The hardware implementation of this study uses the NUCLEO-F401RE microcontroller. This is a microcontroller developed on an ARM Cortex M4 architecture that runs at 84 MHz and has hardware timers, a 12-bit ADC, and I2C.

4.4.3 System Clock Configuration

The system clock was derived from the internal high-speed oscillator (HSI) in association with a phase-locked loop (PLL). The PLL was configured such that the system clock was set around 84 MHz. The APB (Advanced Peripheral Bus) clocks were set up as follows:

1. SYSCLK: 84 MHz
2. APB1: 42 MHz
3. APB2: 84 MHz

This clock configuration ensures sufficient timing resolution of 0.01 seconds and ensures real-time sampling and timer-based execution.

4.4.4 Hardware Timer-Based Sampling

Need for a Hardware Timer

A constant and predictable sampling period is required for real-time emulation. Delays in software code or the execution of the main loop cause jitter and uncertainty in the timing. To achieve a constant sampling rate unaffected by the execution of the main program, a hardware timer was used.

Timer Configuration

The TIM2 timer was configured to operate in an upward counting mode, and this was achieved using the internal clock. The sampling step was based on the relationship between a timer and a clock, and this relationship is given by:

$$T_s = \frac{(PSC + 1)(ARR + 1)}{f_{TIM}} \quad (4.17)$$

In this equation, PSC is the prescaler, ARR is the auto-reload register, and f_{TIM} is the clock frequency used by the timer. In this case, the values used were: The following parameters were selected:

1. Timer clock frequency, $f_{TIM} = 84 \text{ MHz}$
2. Prescaler, $PSC = 8399$
3. Auto-reload register, $ARR = 99$

Substituting these values yields a sampling time of:

$$T_s = 0.01 \text{ s}$$

Thus, a fixed sampling interval of 10 ms was achieved.

4.4.5 ADC Interface for Input Acquisition

ADC1 was configured in independent mode to acquire real-time analog voltage input from a potentiometer connected to pin PA0.

The ADC configuration parameters are summarized as follows:

1. Resolution: 12 bits
2. Conversion mode: Single conversion
3. Trigger source: Software trigger
4. Sampling time: 144 cycles
5. Input voltage range: 0–3.3 V

The ADC output provides a digital representation of the input voltage, which will later be mapped to PMSM model variables such as speed or torque reference.

4.4.6 HAL-Based Programming Approach

All peripherals were accessed through the STM32 Hardware Abstraction Layer (HAL). The HAL provides portability and ease of maintenance by hiding the lower-level details of the registers. The project employed the following HAL modules:

1. HAL_ADC
2. HAL_I2C
3. HAL_GPIO
4. HAL_RCC
5. HAL_TIM

HAL-based programming enabled rapid development and simplified debugging during peripheral integration.

4.4.7 Digital Filter Realization of Discrete PMSM Model

Once the timer-based execution mechanism is in place, the discretized PMSM transfer function is implemented as a difference equation. The choice is a Direct Form-I structure due to its reduced memory requirements and computational efficiency.

The general second-order difference equation implemented is given by:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} \quad (4.18)$$

The corresponding Direct Form-I difference equation is given by:

$$y(k) = -a_1 y(k-1) - a_2 y(k-2) + b_0 u(k) + b_1 u(k-1) + b_2 u(k-2) \quad (4.19)$$

where:

1. $u(k)$ is the input voltage at sample k
2. $y(k)$ is the output speed at sample k
3. a_1, a_2 are feedback coefficients
4. b_0, b_1, b_2 are feedforward coefficients

The state variables were updated during each timer interrupt to ensure deterministic real-time execution.

4.4.8 Interrupt-Driven Execution Architecture

The discrete PMSM computation was executed inside the timer interrupt service routine (ISR). The main loop was intentionally left empty to prevent timing jitter and ensure fixed sampling

```
while(1)
{
    // Real-time execution handled in ISR
}
```

This guarantees:

1. Deterministic sampling
2. Fixed computational cycle
3. Minimum execution jitter

4.4.9 ADC-to-Speed Mapping

The 12-bit ADC provides digital values in the range 0–4095 corresponding to 0–3.3 V input, this input represents the input voltage fed to the PMSM motor

The measured ADC value was mapped to a representative motor speed using:

$$\text{Speed (RPM)} = \frac{\text{ADC}}{4095} \times 4600 \quad (4.20)$$

This mapping enabled real-time visualization of scaled motor speed corresponding to varying potentiometer input.

4.4.10 UART-Based Verification

USART2 was configured at a baud rate of 115200 for real time debugging and validation . Real-time ADC values and computed outputs were transmitted to a serial terminal (PuTTY).

This step confirms:

1. Correct peripheral configuration
2. Stable real-time execution
3. Proper numerical computation of discrete equations

4.4.11 Embedded Implementation Challenges

However, during integration, several issues were encountered:

1. Instability in LCD character display due to timing sensitivity in I2C communication

2. Peripheral handle mismatches, which were encountered during project restructuring
3. The management of floating-point operations during the sampling interval was addressed.

The debugging and tuning of the configuration of the code in STM32CubeIDE resolved the problems. The structured approach is the foundation for the inclusion of the complete DT PMSM model in the future.

This structured methodology forms the foundation for future integration of the complete PMSM discrete-time model.

4.4.12 Hardware Results using UART (PuTTY)

The PMSM emulator has been tested in real-time and validated on actual hardware. The microcontroller reads the analog voltage of the potentiometer through the ADC and then passes it through the discretized model. The output is then sent over the UART and printed on the PuTTY terminal. The output is the ADC value, the input voltage V_{in} , the scaled motor voltage, and the motor speed in RPM. The output is then sent over the UART and printed on the PuTTY terminal as it is. The PuTTY ?? screenshot confirms that it is indeed running in real-time. The output is constantly changing as the input voltage is constantly changing. The key points to note here are:

1. The ADC value is constantly drifting because of the noise in the analog input even if the potentiometer is kept constant.
2. The V_{in} is derived correctly from the ADC value within the 0-3.3 V range.
3. The RPM is derived correctly according to the transfer function model.
4. The output is constantly updating. The output is constantly updating because it is indeed running in real-time on the embedded system.

It is observed that:

1. The ADC values vary slightly due to inherent noise in analog input, even when the potentiometer is kept constant.
2. The input voltage (V_{in}) is correctly mapped from ADC values in the range of 0–3.3V.
3. The emulator generates corresponding speed values in RPM based on the implemented transfer function model.
4. The output updates continuously, demonstrating real-time execution of the embedded system.

Thus, even though there are fluctuations in the output due to ADC noise and discretization, the overall behavior of the output verifies the successful implementation of the PMSM emulator on the STM32 hardware.

The results demonstrate that the emulator is capable of producing motor-like responses and can be used for further analysis and development of motor control systems.

4.4.13 Conclusion

This chapter presented a clear and systematic approach to developing the STM32-based PMSM emulator. It begins with the motor's dynamics, for which there is a strong mathematical model, and upon which a proper understanding of the system's behavior can be based. The chapter then goes on to present various discretization techniques, discussing how the continuous time models can be converted into practical discrete time filters. The choice of Infinite Impulse Response (IIR) filters implemented in Direct Form I was motivated by the need for computational efficiency within the constraints of embedded hardware and the structure of the transfer function .

The process of modeling, discretizing, and embedding the realization ensures that the emulator is faithful in reproducing the motor behavior and is still in real-time on the STM32 microcontroller platform using the Cube IDE.

```
PuTTY (inactive)
ADC:4087 Vin:3.29V Motor:165.6V Speed:3004 RPM
ADC:4085 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4089 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4089 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4089 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4086 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4085 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4083 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:3643 Vin:3.20V Motor:161.0V Speed:3068 RPM
ADC:3925 Vin:3.26V Motor:163.8V Speed:3128 RPM
ADC:3913 Vin:3.15V Motor:158.6V Speed:3004 RPM
ADC:4089 Vin:3.30V Motor:165.8V Speed:3068 RPM
ADC:4091 Vin:3.30V Motor:165.8V Speed:3128 RPM
ADC:4079 Vin:3.29V Motor:165.6V Speed:3004 RPM
ADC:3551 Vin:2.86V Motor:144.1V Speed:3068 RPM
ADC:3559 Vin:2.87V Motor:144.2V Speed:3128 RPM
ADC:3555 Vin:2.87V Motor:144.2V Speed:3004 RPM
ADC:3557 Vin:2.87V Motor:144.2V Speed:3068 RPM
ADC:3503 Vin:2.82V Motor:142.0V Speed:3128 RPM
ADC:3613 Vin:2.87V Motor:144.4V Speed:3004 RPM
ADC:4093 Vin:3.30V Motor:165.9V Speed:3068 RPM
ADC:4095 Vin:3.30V Motor:165.9V Speed:3128 RPM
ADC:4094 Vin:3.30V Motor:165.9V Speed:3004 RPM
ADC:4093 Vin:3.30V Motor:165.9V Speed:3068 RPM
ADC:4095 Vin:3.30V Motor:165.9V Speed:3128 RPM
ADC:4094 Vin:3.30V Motor:165.9V Speed:3004 RPM
ADC:4095 Vin:3.30V Motor:165.9V Speed:3068 RPM
ADC:4051 Vin:3.27V Motor:164.3V Speed:3128 RPM
ADC:3835 Vin:3.09V Motor:155.5V Speed:3004 RPM
ADC:3608 Vin:2.92V Motor:146.9V Speed:3068 RPM
ADC:3401 Vin:2.74V Motor:137.7V Speed:3128 RPM
ADC:3222 Vin:2.60V Motor:130.8V Speed:3004 RPM
ADC:3039 Vin:2.45V Motor:123.5V Speed:3068 RPM
```

Figure 4.5: Real-time UART Output of PMSM Emulator displayed in PuTTY

Chapter 5

Results

5.1 Introduction

This chapter provides a detailed assessment of discretization schemes used in the PMSM emulator, including comparisons of their performance over different sampling intervals. The time-domain errors, i.e., Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Integral of Absolute Error (IAE) defined in (4.10)–(4.13), along with frequency-domain errors, i.e., magnitude error, phase error, and H_∞ norm defined in (4.14)–(4.16), will be used.

5.2 Results

5.2.1 Results at Sampling Time $T_s = 0.001$ s

Time Domain Error Metric

The step response of the system at the chosen sampling interval, both for the continuous system and the discretized system, is illustrated in Figure 5.1. It enables us to compare all the methods visually. It also confirms that all the methods, including ZOH, FOH, Tustin, and Matched Z, follow the continuous system model. The peak of the amplitude is around 1.35, while the settling value is approximately 0.7. In Table 5.1, the most important error criteria of all the discretization methods in the time domain are provided. It shows that the MSE and MAE of the ZOH and the Matched Z methods are the smallest, while the FOH and Tustin methods have a slightly higher, but minor, deviation.

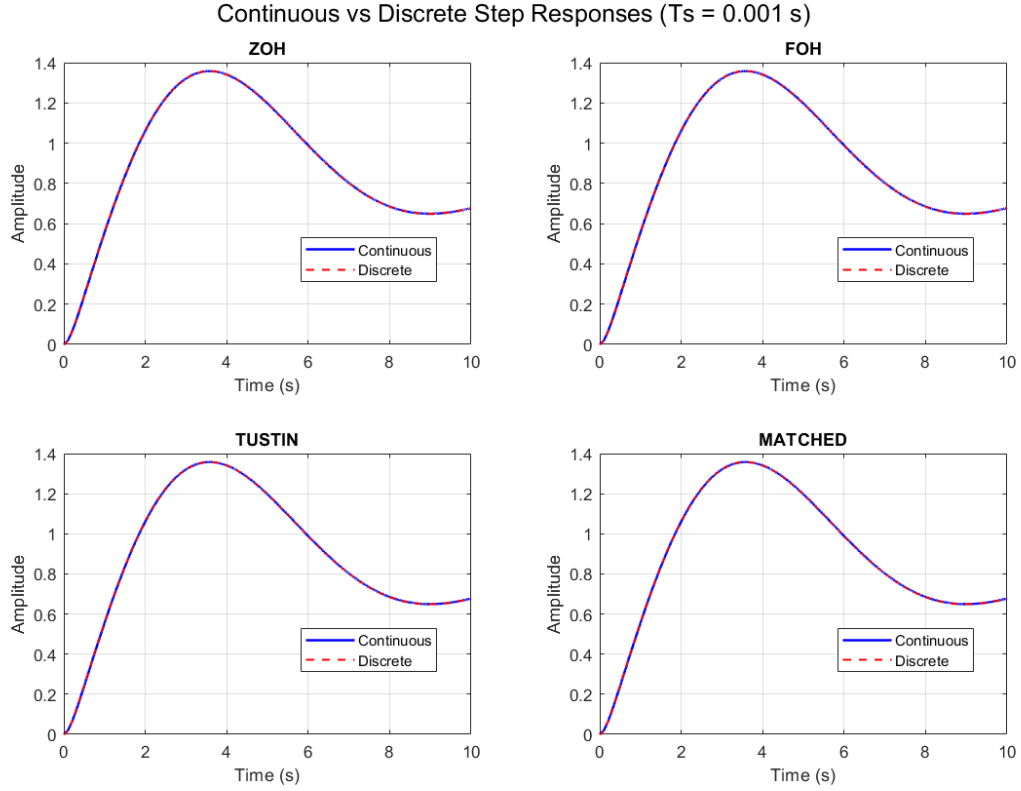


Figure 5.1: Step responses of continuous and discretized systems at $T_s = 0.001$ s.

Table 5.1: Time-domain error metrics at $T_s = 0.001$ s

Method	MSE	RMSE	MAE	IAE
ZOH	1.394×10^{-7}	3.734×10^{-4}	1.462×10^{-4}	1.462×10^{-3}
FOH	1.583×10^{-7}	3.978×10^{-4}	2.125×10^{-4}	2.125×10^{-3}
Tustin	1.583×10^{-7}	3.978×10^{-4}	2.125×10^{-4}	2.125×10^{-3}
Matched Z	1.394×10^{-7}	3.734×10^{-4}	1.462×10^{-4}	1.462×10^{-3}

Frequency Domain Metric Evaluation

The key metrics of the frequency response are presented in Table 5.2. Tustin has the least phase error of 0.017° . FOH has the least H error, which is indicative of high magnitude response fidelity. All the methods have good digital stability as their largest pole radius is well below 1.0 (0.99979)

Table 5.2: Frequency-domain metrics at $T_s = 0.001$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	2.596×10^{-6}	15.048	6.262×10^{-4}	0.99979
2	FOH	4.857×10^{-6}	3.466×10^{-5}	3.921×10^{-7}	0.99979
3	Tustin	8.707×10^{-6}	0.017	3.165×10^{-8}	0.99979
4	Matched Z	3.401×10^{-6}	15.066	6.275×10^{-4}	0.99979

Trends at $T_s = 0.001$ s

The summary plots for the sampling time $T_s = 0.001$ s, including step responses, error signals, Bode magnitude/phase plots, and pole-zero maps are shown in Figure 5.2.

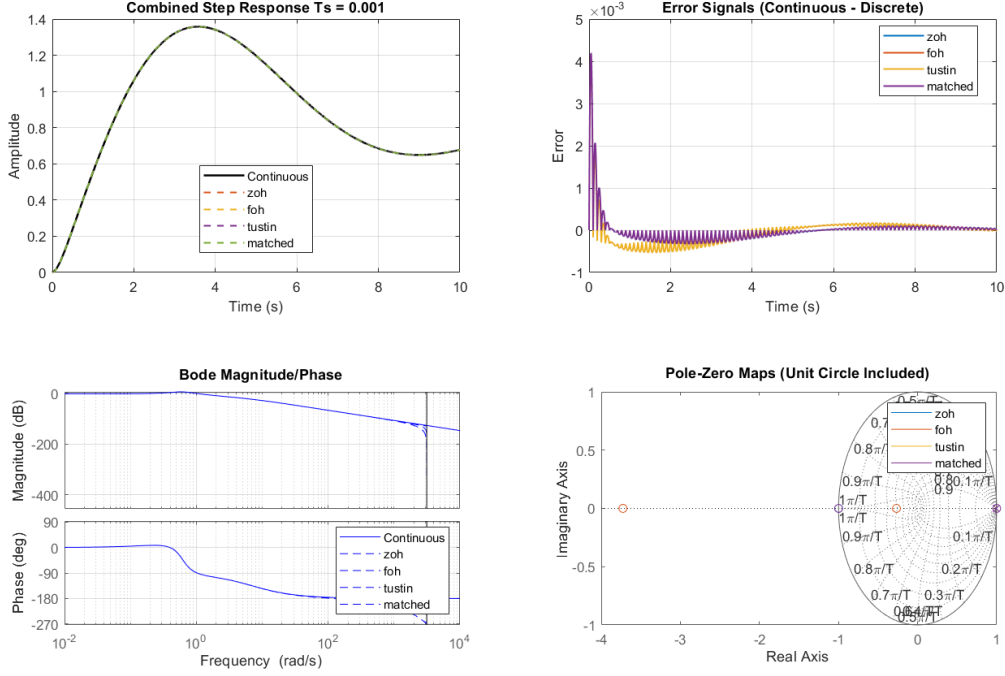


Figure 5.2: General trends for $T_s = 0.001$ s: step responses, error signals, Bode plots, and pole-zero map.

Summary

From these results, it is possible to conclude that at $T_s = 0.001$ s, all discretization techniques provide a very precise performance in the time and frequency domains in relation to the continuous PMSM dynamics, and Zero Order Hold and Matched Z are the most precise techniques among the ones analyzed.

5.2.2 Results at Sampling Time $T_s = 0.01$ s

Time Domain Error Metric

Figure 5.3 depicts the step response comparison between the continuous reference and discrete approximations obtained via ZOH, FOH, Tustin, and Matched Z-transform methods at $T_s = 0.01$ s. As expected, with the coarser sampling interval, minor deviations begin to appear in transient response characteristics such as peak amplitude and settling time, yet the overall shape remains closely approximated.

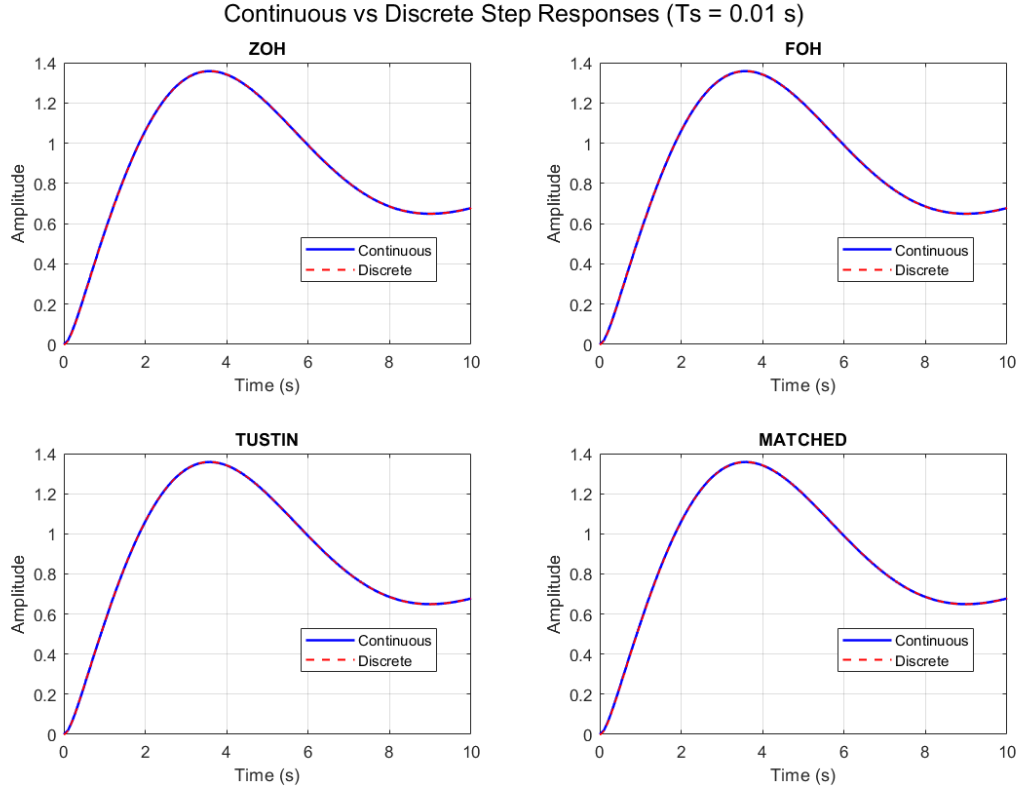


Figure 5.3: Step responses for continuous and discretized systems at $T_s = 0.01$ s.

Table 5.3 summarizes the time-domain error metrics at this sampling interval. The ZOH method demonstrates the lowest errors overall, though increases from finer sampling are evident, particularly in MAE and IAE for FOH and Tustin methods.

Table 5.3: Time-domain error metrics at $T_s = 0.01$ s

Method	MSE	RMSE	MAE	IAE
ZOH	4.783×10^{-6}	2.187×10^{-3}	1.195×10^{-3}	1.195×10^{-2}
FOH	1.201×10^{-5}	3.465×10^{-3}	2.933×10^{-3}	2.933×10^{-2}
Tustin	1.518×10^{-5}	3.898×10^{-3}	2.991×10^{-3}	2.991×10^{-2}
Matched Z	5.044×10^{-6}	2.245×10^{-3}	1.230×10^{-3}	1.230×10^{-2}

Frequency Domain Metric Evaluation

Frequency-domain metrics at $T_s = 0.01$ s reveal increased magnitude and phase errors as sampling coarsens compared to $T_s = 0.001$ s, as shown in Table 5.4. ZOH and Matched Z maintain the smallest magnitude error and robust stability, with maximum pole radius well within unit bounds.

Table 5.4: Frequency-domain metrics at $T_s = 0.01$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	3.089×10^{-4}	6.468	1.240×10^{-2}	0.99947
2	FOH	6.391×10^{-4}	18.129	3.677×10^{-2}	0.99944
3	Tustin	2.435×10^{-4}	7.012	9.338×10^{-3}	0.99927
4	Matched Z	3.371×10^{-4}	6.983	1.352×10^{-2}	0.99948

General Trends at $T_s = 0.01$ s

Figure 5.4 presents the combined analysis summarizing step response, temporal error evolution, frequency response, and pole-zero map for this sampling interval.

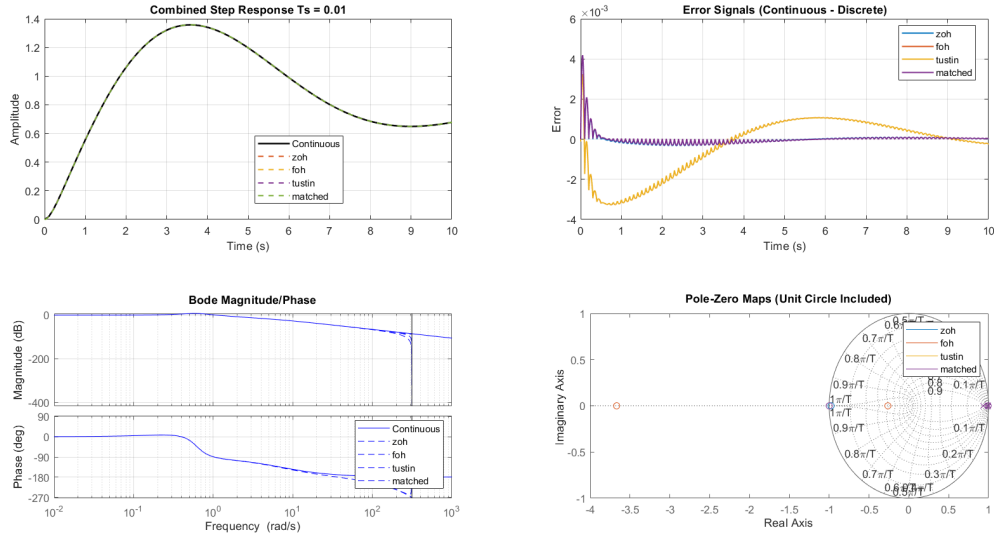


Figure 5.4: Summary plots for $T_s = 0.01$ s: step responses, error signals, Bode plots, and pole-zero map.

Summary

Sampling time $T_s = 0.01$ s introduces slightly greater deviations compared to finer sampling but maintains acceptable accuracy and stability for practical control applications. The ZOH and Matched Z methods remain preferred for balancing accuracy with computational efficiency.

5.2.3 Results at Sampling Time $T_s = 0.1$ s

Time Domain Error Metric

Figure 5.5 shows a comparison of the step response of the continuous reference system with its discrete counterparts obtained using ZOH, FOH, Tustin, and the Matched Z-transform with a sampling interval of $T_s = 0.1$ s. The deviations from the continuous response are evident, and this indicates that with a coarser sampling rate, the difference between the discrete and continuous response will be larger. Caption for Figure 5.5: Step response of the continuous and

discrete system with $T_s = 0.1$ s. Table 5.5 describes the error values for the discrete system. The Zero-Order Hold (ZOH) method keeps the error values relatively lower, whereas FOH and Tustin show much larger error values.

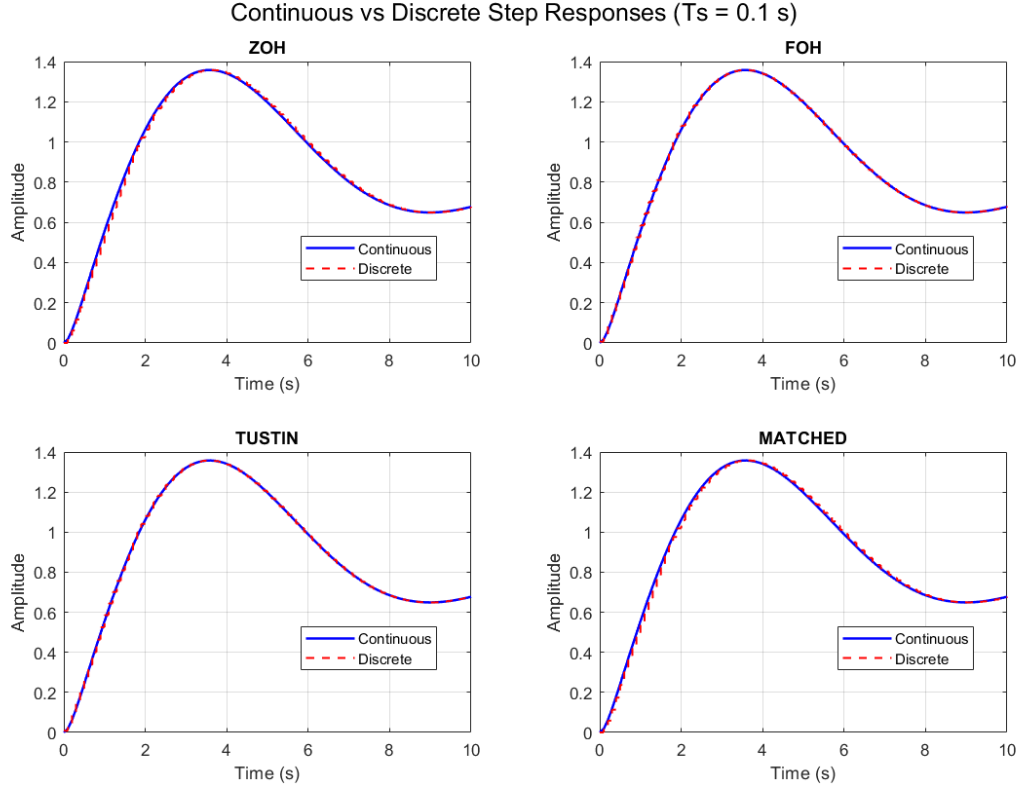


Figure 5.5: Step responses at $T_s = 0.1$ s.

As presented in Table 5.5, the time-domain error metrics are as follows: the Zero Order Hold (ZOH) technique maintains low error values, whereas the FOH and Tustin methods have significantly higher MSE, RMSE, MAE, and IAE values, reflecting a decline in approximation error

Table 5.5: Time-domain error metrics at $T_s = 0.1$ s

Method	MSE	RMSE	MAE	IAE
ZOH	5.342×10^{-4}	2.310×10^{-2}	1.228×10^{-2}	1.228×10^{-1}
FOH	2.844×10^{-2}	1.686×10^{-1}	1.071×10^{-1}	1.071×10^0
Tustin	2.538×10^{-2}	1.592×10^{-1}	9.863×10^{-2}	9.863×10^{-1}
Matched Z	6.092×10^{-4}	2.468×10^{-2}	1.313×10^{-2}	1.313×10^{-1}

Frequency Domain Metric Evaluation

In Table 5.6, the frequency domain is considered. The error in the magnitude and phase for the FOH and Tustin methods increases significantly, with errors up to two orders of magnitude greater than those of ZOH and Matched Z. However, all poles remain within the unit circle.

Table 5.6: Frequency-domain metrics at $T_s = 0.1$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	3.443×10^{-3}	43.578	1.292×10^{-1}	0.99802
2	FOH	1.102×10^{-1}	232.336	1.748×10^0	0.97976
3	Tustin	1.261×10^{-1}	196.224	1.066×10^0	0.97094
4	Matched Z	3.507×10^{-3}	44.084	1.348×10^{-1}	0.99807

General Trends at $T_s = 0.1$ s

Figure 5.6 summarizes the step responses, error evolution over time, frequency response, and pole-zero stability plot for sampling time $T_s = 0.1$ s.

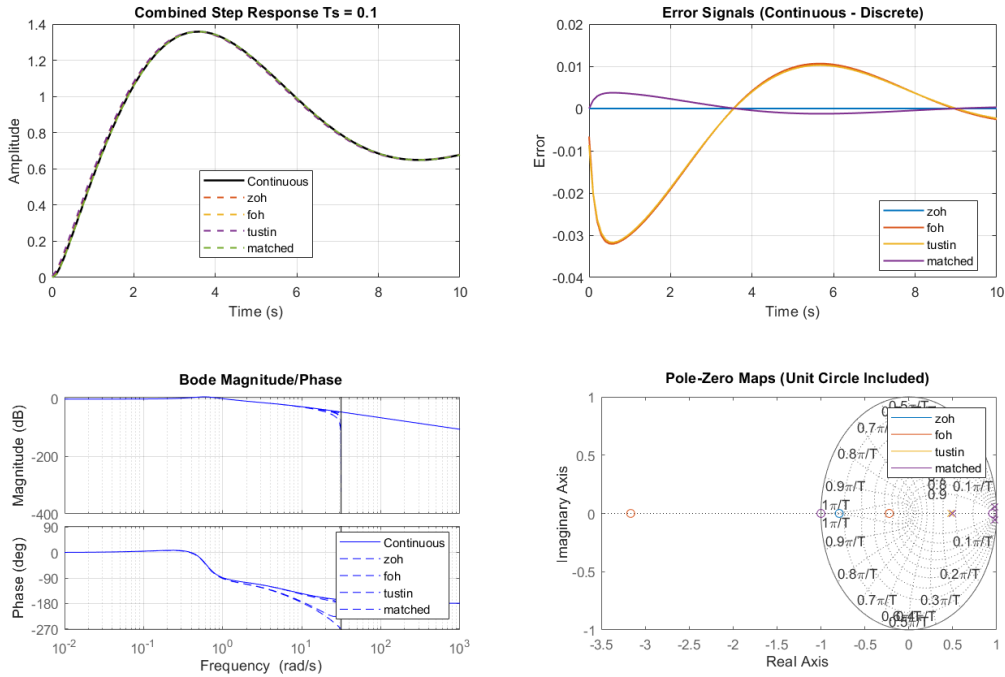


Figure 5.6: Summary plots for $T_s = 0.1$ s: step responses, error signals, Bode plots, and pole-zero map.

Summary

At the relatively coarse sampling interval of $T_s = 0.1$ s, the ZOH and Matched Z methods continue to exhibit modest errors and retain system stability, while FOH and Tustin experience significantly larger deviations that could affect high-precision applications. These results provide insight into the trade-offs involved in discretization and sampling choices for PMSM embedded emulators.

5.2.4 Results at Sampling Time $T_s = 1$ s

Time Domain Error Metric

Figure 5.7 depicts the step responses of the continuous system, as well as the discretized system with $T_s=1$ s. In this case, the difference between the system's transient and steady-state responses, as well as the slower rates of rise, are evident when the system has a coarser discretization, as seen with the FOH and Tustin methods.

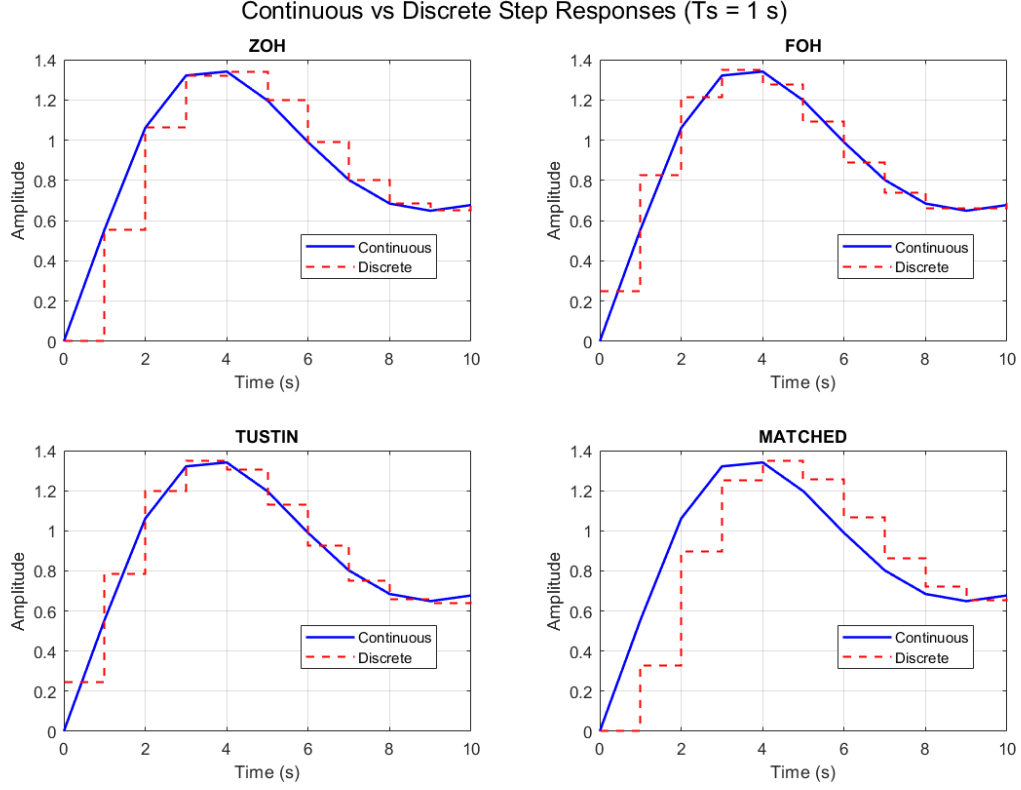


Figure 5.7: Step responses for continuous and discretized systems at $T_s = 1$ s.

The time domain error parameters, i.e., MSE and RMSE, are listed in Table 5.7. The values are larger than in the previous case, and for FOH and Tustin, they are significantly larger, which indicates a decrease in the accuracy of the approximation.

Table 5.7: Time-domain error metrics at $T_s = 1$ s

Method	MSE	RMSE	MAE	IAE
ZOH	3.742×10^{-2}	1.934×10^{-1}	1.210×10^{-1}	1.210
FOH	8.619×10^{-2}	2.935×10^{-1}	2.164×10^{-1}	2.164
Tustin	9.837×10^{-2}	3.137×10^{-1}	2.311×10^{-1}	2.311
Matched Z	4.009×10^{-2}	2.002×10^{-1}	1.234×10^{-1}	1.234

Frequency Domain Metric Evaluation

Table 5.8 details the frequency-domain metrics. Magnitude and phase errors peak notably here, especially for FOH and Tustin. However, all methods maintain poles within the unit circle (max radius 0.973), preserving stability.

Table 5.8: Frequency-domain metrics at $T_s = 1$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	2.611×10^{-2}	39.134	1.095×10^0	0.96789
2	FOH	1.559×10^{-1}	123.492	7.766×10^0	0.97303
3	Tustin	1.604×10^{-1}	93.009	4.031×10^0	0.96975
4	Matched Z	2.691×10^{-2}	41.177	1.119×10^0	0.96815

General Trends at $T_s = 1$ s

Figure 5.8 summarizes frequency domain performance metrics analysis with step responses, error evolution, frequency response, and pole-zero plots for $T_s = 1$ s .

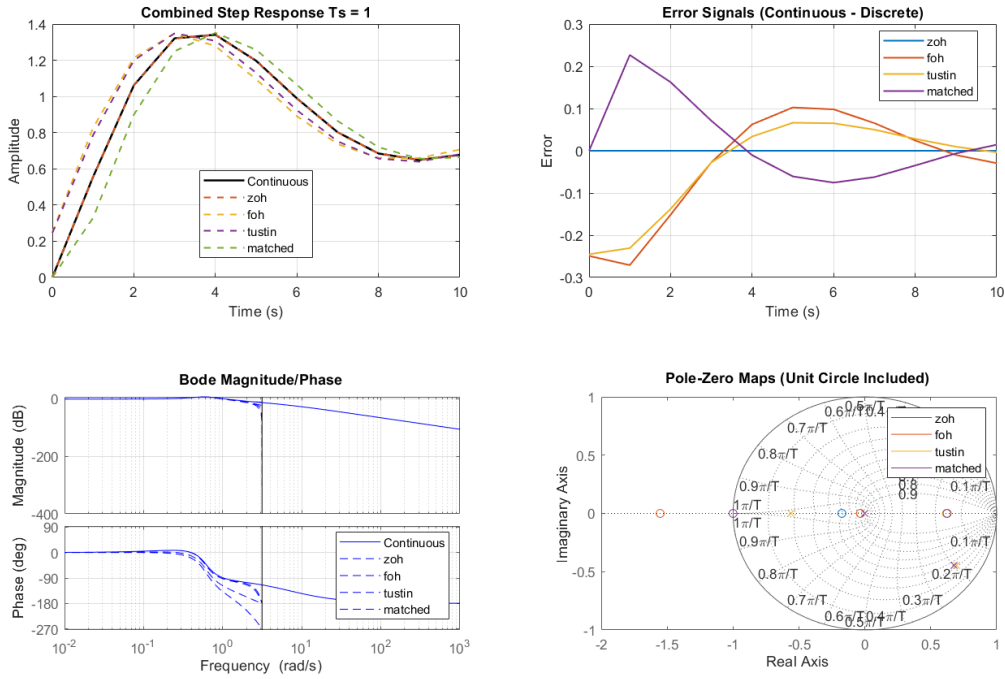


Figure 5.8: Summary plots for $T_s = 1$ s.

Summary

In view of the low resolution of the sampling, the discretization methods introduce more prominent approximation error and phase distortion, which might limit the accuracy of these methods in precision control of dynamic systems. However, ZOH and Matched Z are the more accurate methods.

5.2.5 Results at Sampling Time $T_s = 2$ s

Time Domain Error Metric

The step responses for the continuous reference and discrete system with $T_s = 2$ s are shown in Figure 5.9. With this fairly coarse sampling, differences in the transient as well as steady-state responses can be noted.

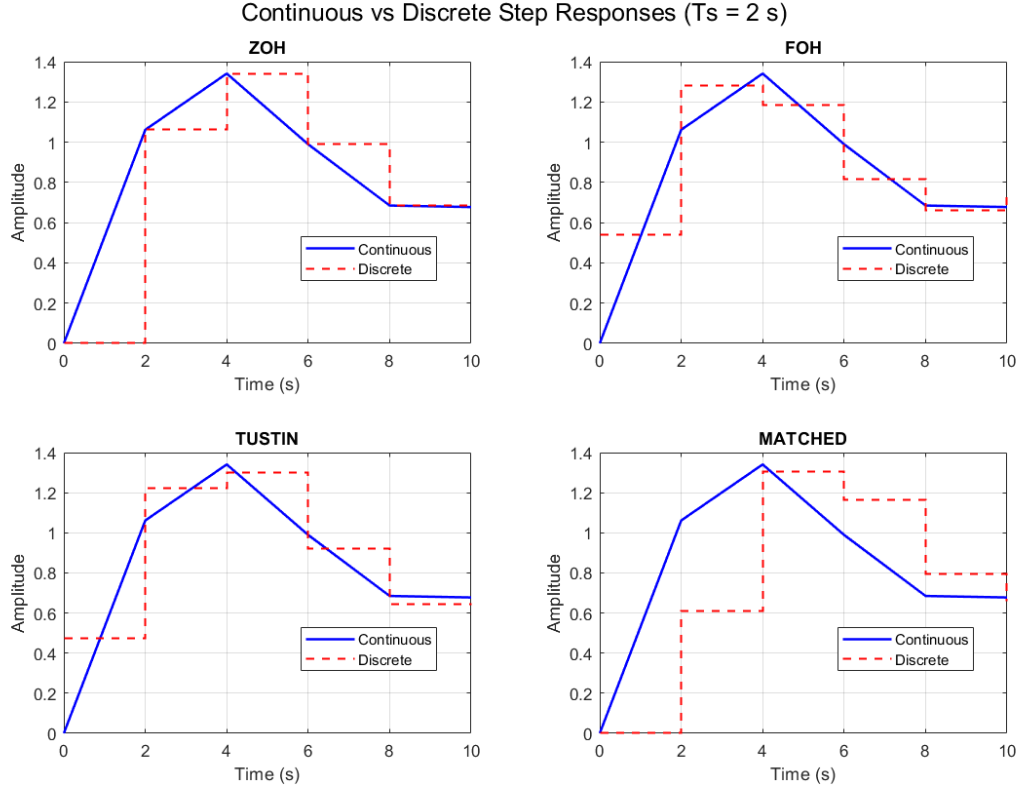


Figure 5.9: Step responses for continuous and discretized systems at $T_s = 2$ s.

Table 5.9 presents corresponding time-domain error metrics. Errors are significantly larger compared to finer sampling intervals, with ZOH and Matched Z performing better than FOH and Tustin.

Table 5.9: Time-domain error metrics at $T_s = 2$ s

Method	MSE	RMSE	MAE	IAE
ZOH	0.2963	0.5444	0.3917	3.9173
FOH	0.9446	0.9720	0.5581	5.5811
Tustin	0.7198	0.8484	0.4706	4.7059
Matched	0.3377	0.5802	0.4081	4.0806

Frequency Domain Metric Evaluation

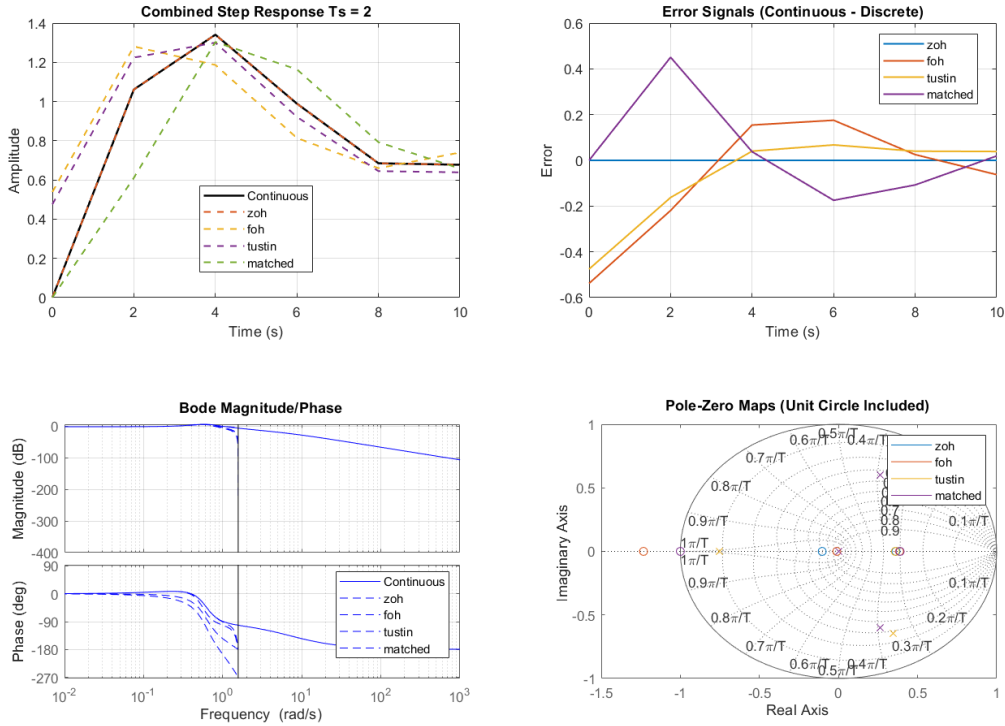
Magnitude and phase errors increase significantly, especially for FOH and Tustin as shown in Table 5.10. All methods maintain poles inside the unit circle, confirming stability.

Table 5.10: Frequency-domain metrics at $T_s = 2$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	2.108×10^{-2}	37.195	1.792×10^0	0.89828
2	FOH	1.287×10^{-1}	453.131	1.105×10^1	0.90131
3	Tustin	1.293×10^{-1}	395.101	2.010×10^0	0.89539
4	Matched Z	2.519×10^{-2}	39.860	1.890×10^0	0.89931

General Trends at $T_s = 2$ s

Figure 5.10 summarizes step response, error signal, Bode plots, and pole-zero maps at $T_s = 2$ s.


 Figure 5.10: Performance for $T_s = 2$ s.

Summary

In the coarse time step of 2 seconds, the discretization error increases, and this causes deviations in the emulation of the PMSM's dynamics. However, all discretization techniques are found to be stable, with ZOH and Matched Z having high accuracy.

5.2.6 Results at Sampling Time $T_s = 3$ s

Time Domain Error Metric

As shown in Figure 5.11, the step responses of continuous and discrete PMSM models are illustrated when the sampling period is $T_s = 3$ s. It is observed that due to a lower sampling period,

the step response is significantly affected for both FOH and Tustin discretization schemes.

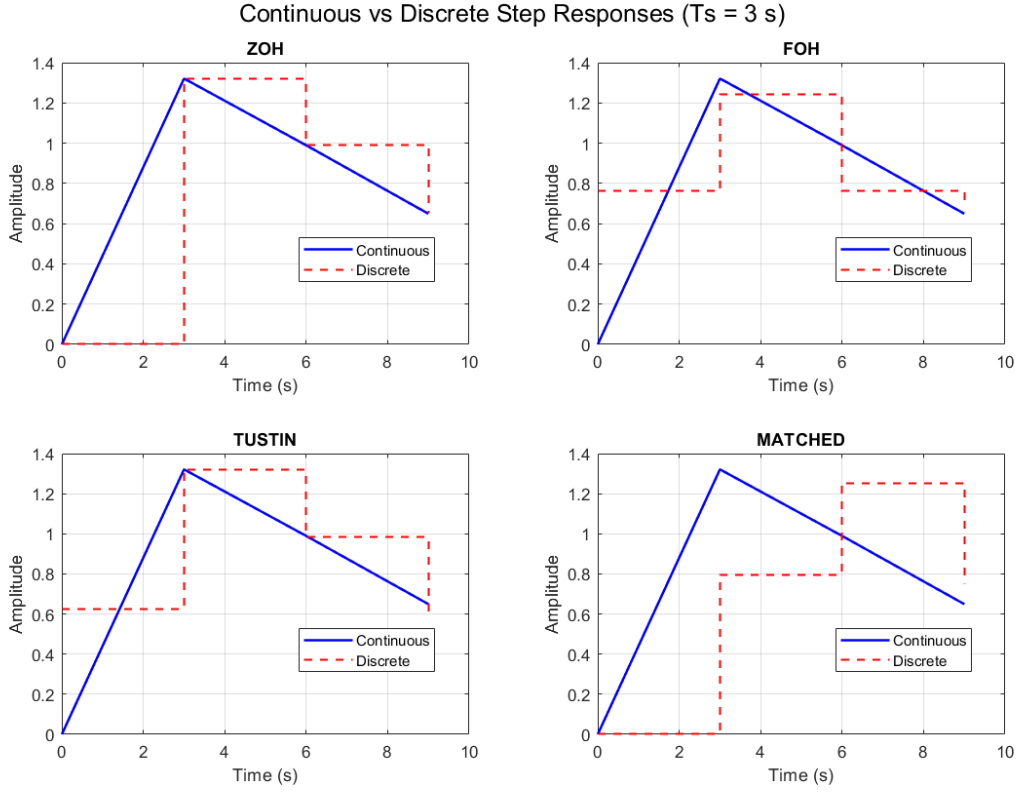


Figure 5.11: Step responses for continuous and discretized systems at $T_s = 3$ s.

The time domain results are given in Table 5.11. The MSE for FOH and Tustin methods approaches or exceeds 0.3, while ZOH and Matched Z methods demonstrate better handling in error growth, although all methods degrade with finer sampling.

Table 5.11: Time-domain error metrics at $T_s = 3$ s

Method	MSE	RMSE	MAE	IAE
ZOH	0.1328	0.3643	0.2937	2.9373
FOH	0.3429	0.5856	0.4461	4.4611
Tustin	0.3026	0.5500	0.3771	3.7713
Matched	0.1481	0.3848	0.3114	3.1142

Frequency Domain Metric Evaluation

The frequency domain metrics in Table 5.12 further highlight additional magnitude and phase errors, but discrete time stability is maintained with all poles inside the unit circle, with a maximum radius of 0.85.

Table 5.12: Frequency-domain metrics at $T_s = 3$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	1.207×10^{-2}	52.511	7.553×10^{-1}	0.85347
2	FOH	2.273×10^{-1}	324.483	3.995×10^0	0.84812
3	Tustin	1.990×10^{-1}	232.397	2.131×10^0	0.85652
4	Matched Z	1.493×10^{-2}	53.224	8.592×10^{-1}	0.85111

General Trends at $T_s = 3$ s

Figure 5.12 visualizes the combined analysis of time and frequency characteristics including step responses, error evolution, Bode plots, and pole-zero maps.

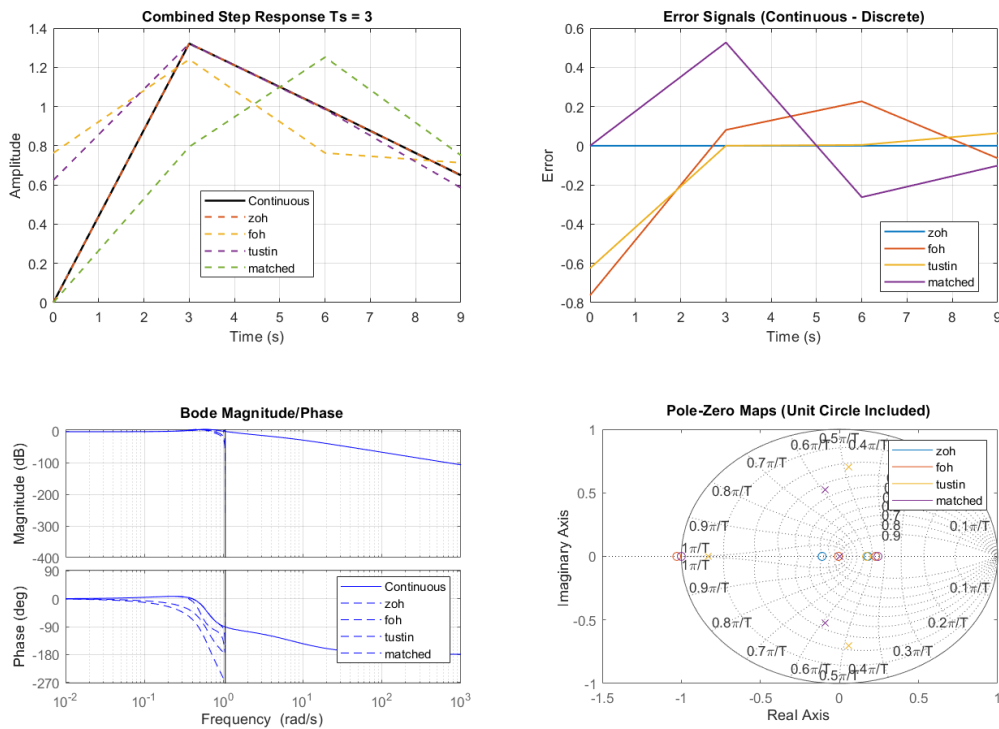


Figure 5.12: Summary plots for $T_s = 3$ s: step responses, error signals, Bode plots, and pole-zero map.

Summary

With $T_s = 3$ s, marked degradation in dynamic fidelity occurs, reflected in large errors and phase distortions. Nonetheless, all methods remain stable with poles well inside the stability boundary. ZOH and Matched Z methods exhibit relatively better robustness under coarse sampling.

5.2.7 Results at Sampling Time $T_s = 5$ s

Time Domain Error Metric

Figure 5.13 shows the step responses of continuous and discretized systems at $T_s = 5$ s. Noticeable degradation in transient response and steady-state accuracy occurs due to very coarse sampling, especially in FOH and Tustin methods.

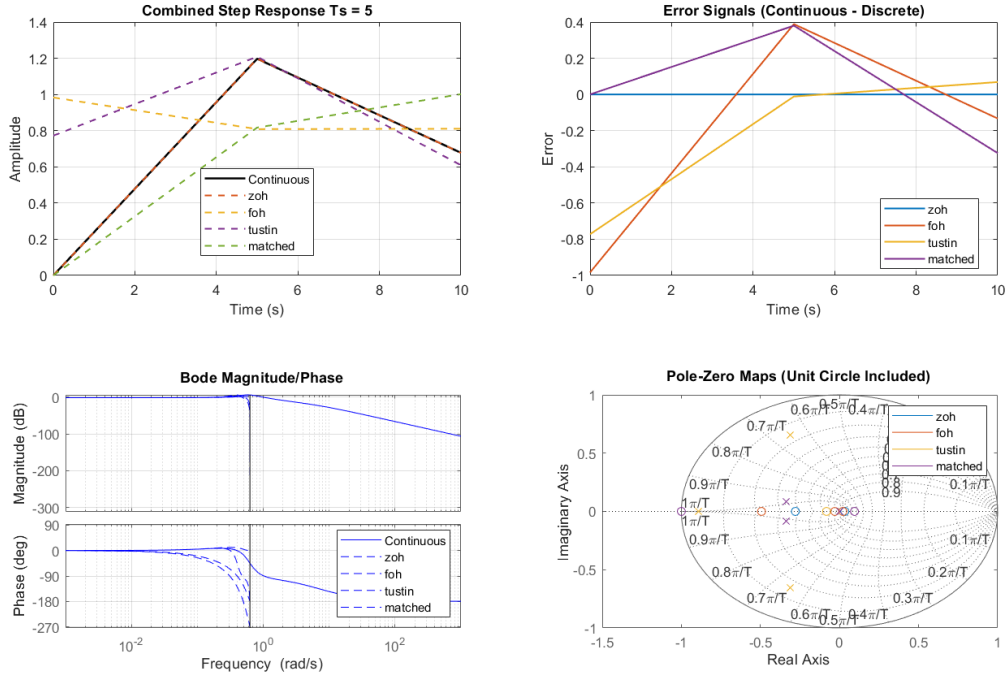


Figure 5.13: Step responses for continuous and discretized systems at $T_s = 5$ s.

Table 5.13 provides time-domain errors with substantially increased values relative to finer sampling, demonstrating approximations degrade with sampling coarseness.

Table 5.13: Time-domain error metrics at $T_s = 5$ s

Method	MSE	RMSE	MAE	IAE
ZOH	0.8842	0.9403	0.6346	6.346
FOH	4.6669	2.1613	1.2733	12.733
Tustin	3.0704	1.7514	1.1079	11.079
Matched	1.1206	1.0585	0.6607	6.607

Frequency Domain Metric Evaluation

Table 5.14 gives frequency-domain metrics. Magnitude and phase errors increase markedly, particularly for FOH and Tustin. Poles remain inside the unit circle (0.83 radius), so digital stability holds.

Table 5.14: Frequency-domain metrics at $T_s = 5$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	5.823×10^{-2}	51.839	2.383×10^0	0.83234
2	FOH	1.212×10^0	698.719	1.955×10^1	0.81379
3	Tustin	6.484×10^{-1}	498.753	5.864×10^0	0.83519
4	Matched Z	7.147×10^{-2}	54.783	2.602×10^0	0.83556

General Trends at $T_s = 5$ s

Figure 5.14 presents combined step response, error, frequency response, and pole-zero map.

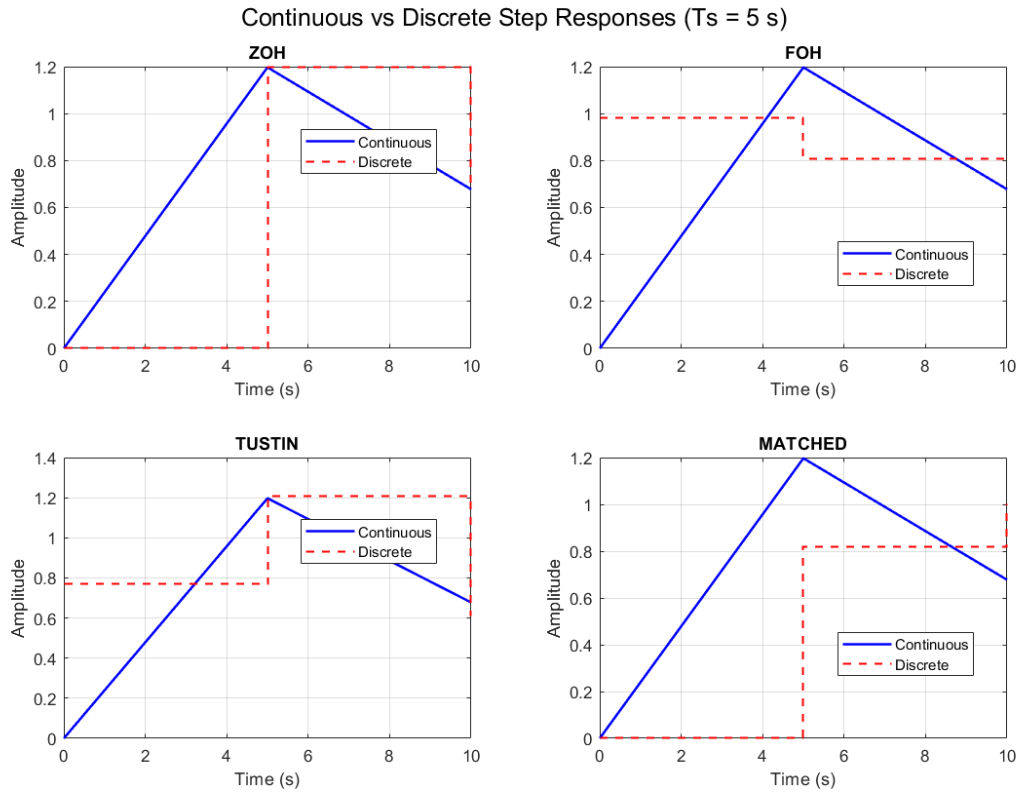


Figure 5.14: Summary plots for $T_s = 5$ s: step responses, error signals, Bode plots, and pole-zero map.

Summary

Sampling time $T_s = 5$ s demonstrates significant loss in discretization fidelity and dynamic accuracy, with elevated errors and phase shifts. While all models remain stable, FOH and Tustin accuracy deteriorates substantially. ZOH and Matched Z continue providing relatively better trade-offs under very coarse sampling.

5.2.8 Results at Sampling Time $T_s = 10$ s

Time Domain Error Metric

Figure 5.15 shows the step responses of continuous and discretized PMSM models at $T_s = 10$ s. Significant deviations are seen, particularly in FOH and Tustin methods, with prolonged settling and shape distortion.

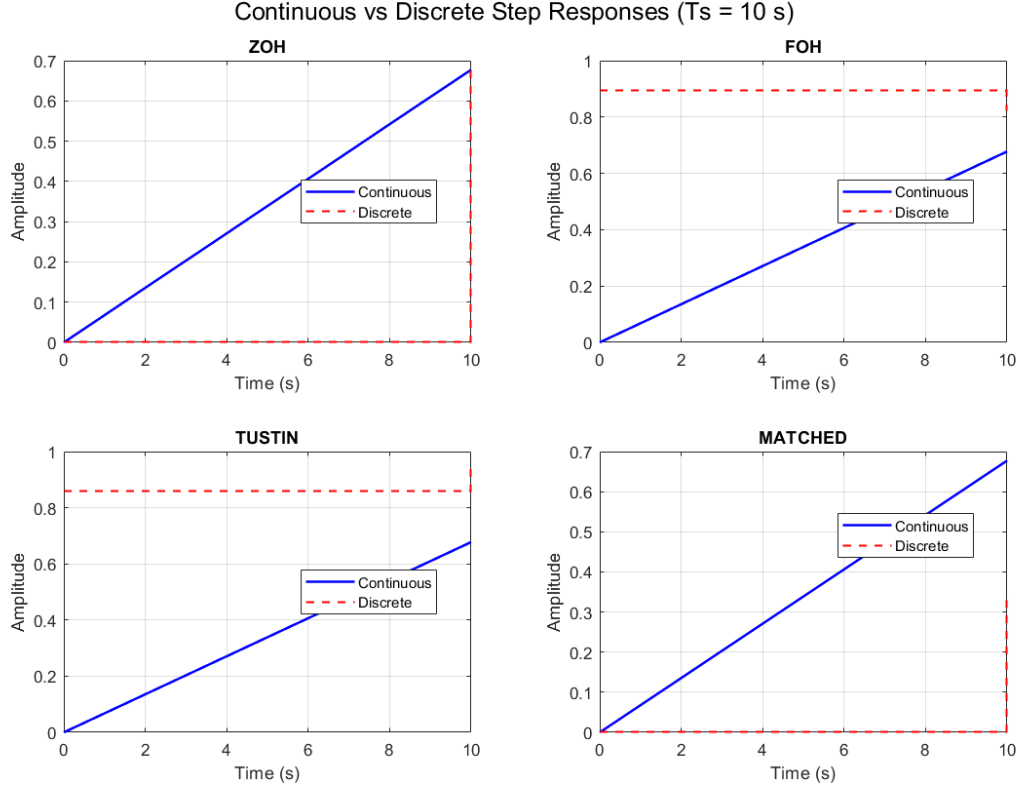


Figure 5.15: Step responses for continuous and discretized systems at $T_s = 10$ s.

Table 5.15 details the time-domain error metrics showing continued rise in error magnitudes, reflecting substantial loss of accuracy for coarser sampling intervals.

Table 5.15: Time-domain error metrics at $T_s = 10$ s

Method	MSE	RMSE	MAE	IAE
ZOH	3.083	1.756	1.214	12.142
FOH	10.854	3.295	2.414	24.141
Tustin	8.661	2.943	2.033	20.330
Matched	3.843	1.961	1.326	13.260

Frequency Domain Metric Evaluation

Table 5.16 shows frequency-domain metrics with large magnitude and phase errors especially in FOH and Tustin, while all poles remain inside the unit circle (max radius 0.76), ensuring

stability at this coarse sampling.

Table 5.16: Frequency-domain metrics at $T_s = 10$ s

Sl. No	Method	Magnitude Error (dB)	Phase Error (deg)	H_∞ Error	Max. Pole Radius
1	ZOH	0.172	76.584	14.214	0.76412
2	FOH	1.129	756.403	189.785	0.75132
3	Tustin	0.908	547.260	72.332	0.76224
4	Matched Z	0.220	82.662	18.512	0.75874

General Trends at $T_s = 10$ s

Figure 5.16 summarizes time and frequency domain performance at $T_s = 10$ s, illustrating deteriorated signal tracking, higher errors, and wider phase distortions.

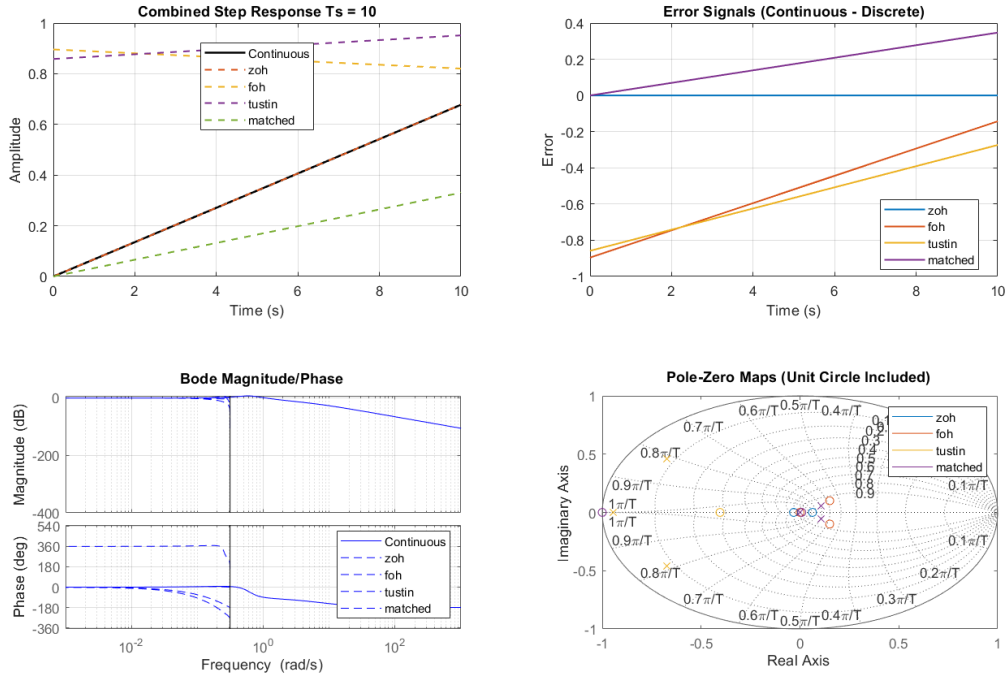


Figure 5.16: Summary plots for $T_s = 10$ s: step responses, error signals, Bode plots, and pole-zero map.

Summary

At the very coarse sampling interval $T_s = 10$ s, discretization errors and latency effects dominate, leading to significant inaccuracies. ZOH and Matched Z continue to outperform FOH and Tustin but coarse sampling greatly limits practical dynamic emulation accuracy.

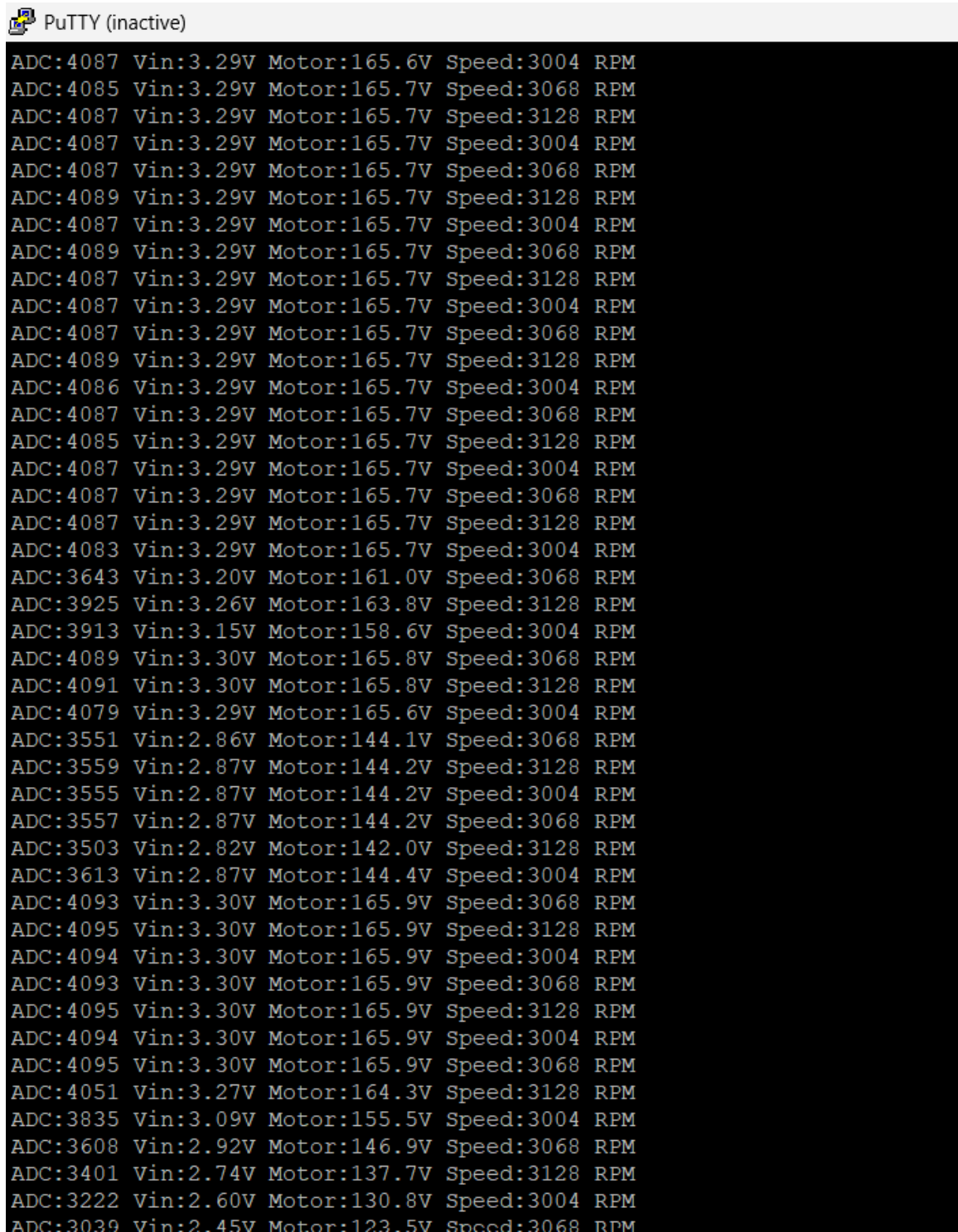
5.3 Embedded Validation Results

This section presents the results obtained from the STM32-based PMSM emulator implementation.

5.3.1 UART-Based Validation

USART2 was configured to verify real time execution at a baud rate of 115200. The ADC readings were transmitted continuously to a serial terminal (PuTTY).

The observed UART output is shown in Fig. 5.17.



```
PuTTY (inactive)
ADC:4087 Vin:3.29V Motor:165.6V Speed:3004 RPM
ADC:4085 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4089 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4089 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4089 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4086 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4085 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3068 RPM
ADC:4087 Vin:3.29V Motor:165.7V Speed:3128 RPM
ADC:4083 Vin:3.29V Motor:165.7V Speed:3004 RPM
ADC:3643 Vin:3.20V Motor:161.0V Speed:3068 RPM
ADC:3925 Vin:3.26V Motor:163.8V Speed:3128 RPM
ADC:3913 Vin:3.15V Motor:158.6V Speed:3004 RPM
ADC:4089 Vin:3.30V Motor:165.8V Speed:3068 RPM
ADC:4091 Vin:3.30V Motor:165.8V Speed:3128 RPM
ADC:4079 Vin:3.29V Motor:165.6V Speed:3004 RPM
ADC:3551 Vin:2.86V Motor:144.1V Speed:3068 RPM
ADC:3559 Vin:2.87V Motor:144.2V Speed:3128 RPM
ADC:3555 Vin:2.87V Motor:144.2V Speed:3004 RPM
ADC:3557 Vin:2.87V Motor:144.2V Speed:3068 RPM
ADC:3503 Vin:2.82V Motor:142.0V Speed:3128 RPM
ADC:3613 Vin:2.87V Motor:144.4V Speed:3004 RPM
ADC:4093 Vin:3.30V Motor:165.9V Speed:3068 RPM
ADC:4095 Vin:3.30V Motor:165.9V Speed:3128 RPM
ADC:4094 Vin:3.30V Motor:165.9V Speed:3004 RPM
ADC:4093 Vin:3.30V Motor:165.9V Speed:3068 RPM
ADC:4095 Vin:3.30V Motor:165.9V Speed:3128 RPM
ADC:4094 Vin:3.30V Motor:165.9V Speed:3004 RPM
ADC:4095 Vin:3.30V Motor:165.9V Speed:3068 RPM
ADC:4051 Vin:3.27V Motor:164.3V Speed:3128 RPM
ADC:3835 Vin:3.09V Motor:155.5V Speed:3004 RPM
ADC:3608 Vin:2.92V Motor:146.9V Speed:3068 RPM
ADC:3401 Vin:2.74V Motor:137.7V Speed:3128 RPM
ADC:3222 Vin:2.60V Motor:130.8V Speed:3004 RPM
ADC:3039 Vin:2.45V Motor:123.5V Speed:3068 RPM
```

Figure 5.17: Real-time UART outputs from STM32-based PMSM emulator.

The ADC readings remain stable with minor variations caused by analog input noise. This confirms:

1. Correct ADC configuration
2. Proper timer-based sampling
3. Successful UART transmission

4. Stable embedded execution

5.4 PMSM Emulator User Interface and Analysis

5.4.1 Overview

For this purpose, a web-based UI using React was created, which simulates the Permanent Magnet Synchronous Motor (PMSM) system. Users can interact with this interface to understand both continuous and discretized versions of this system. The UI integrates:

1. Continuous time transfer function input
2. Discretization using different discretization methods
3. Time-domain and frequency-domain analysis
4. Evaluation metrics computation

5.4.2 Continuous and Discrete Model Input

The user inputs the transfer function in polynomial form:

$$G(s) = \frac{4.705s + 2.219}{s^3 + 7.504s^2 + 3.36s + 2.702} \quad (5.1)$$

The sampling time is selected as:

$$T_s = 0.01 \text{ seconds} \quad (5.2)$$

The system is discretized using the Zero-Order Hold (ZOH) method.

5.4.3 Discrete System Representation

The resulting discrete transfer function is represented as:

$$G(z) = \frac{b_0 + b_1z^{-1} + b_2z^{-2} + b_3z^{-3}}{1 + a_1z^{-1} + a_2z^{-2} + a_3z^{-3}} \quad (5.3)$$

where,

$$\begin{aligned} b &= [-0.2285, 0.6669, -0.6486, 0.2102] \\ a &= [1, -3.0271, 3.0544, -1.0274] \end{aligned}$$

5.4.4 Step Response Analysis

The UI compares continuous and discrete step responses.

1. Continuous response shows smooth behavior.
2. Discrete response exhibits staircase characteristics due to sampling.

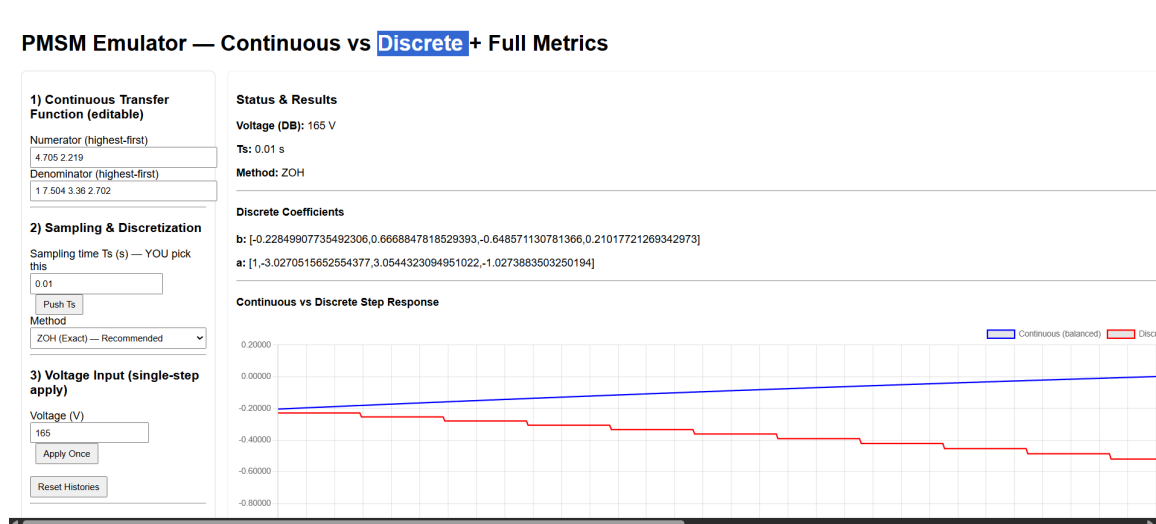


Figure 5.18: Continuous vs Discrete Step Response

5.4.5 Frequency Response (Bode Plot)

The magnitude and phase responses are plotted separately.

1. Continuous system shows gradual attenuation.
2. Discrete system introduces approximation errors at higher frequencies.

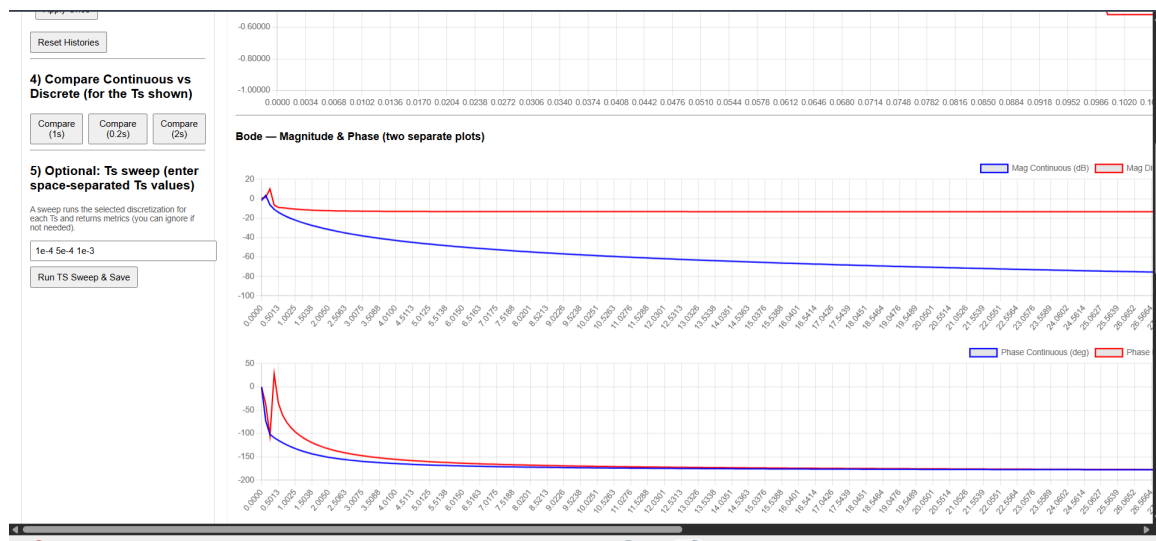


Figure 5.19: Bode Magnitude and Phase Comparison

5.4.6 Performance Metrics

The system computes various error metrics between continuous and discrete responses:

1. Mean Squared Error (MSE): 0.33859
2. Root Mean Squared Error (RMSE): 0.58188
3. Mean Absolute Error (MAE): 0.50361
4. Integral Absolute Error (IAE): 0.10072
5. Integral Squared Error (ISE): 0.06772

The steady-state error is approximately:

$$e_{ss} = 1.00611 \quad (5.4)$$

5.4.7 Stability Analysis

Jury stability test is performed on the discrete system:

1. Jury Test Result: PASSED
2. Maximum pole radius: 1.008111

This indicates that the system is marginally stable.

5.4.8 Sampling Time Sweep

A sweep of different sampling times is performed:

1. $T_s = 0.0001$: MSE = 0.07516
2. $T_s = 0.0005$: MSE = 0.07575
3. $T_s = 0.001$: MSE = 0.07610

Smaller sampling times result in better approximation of the continuous system.

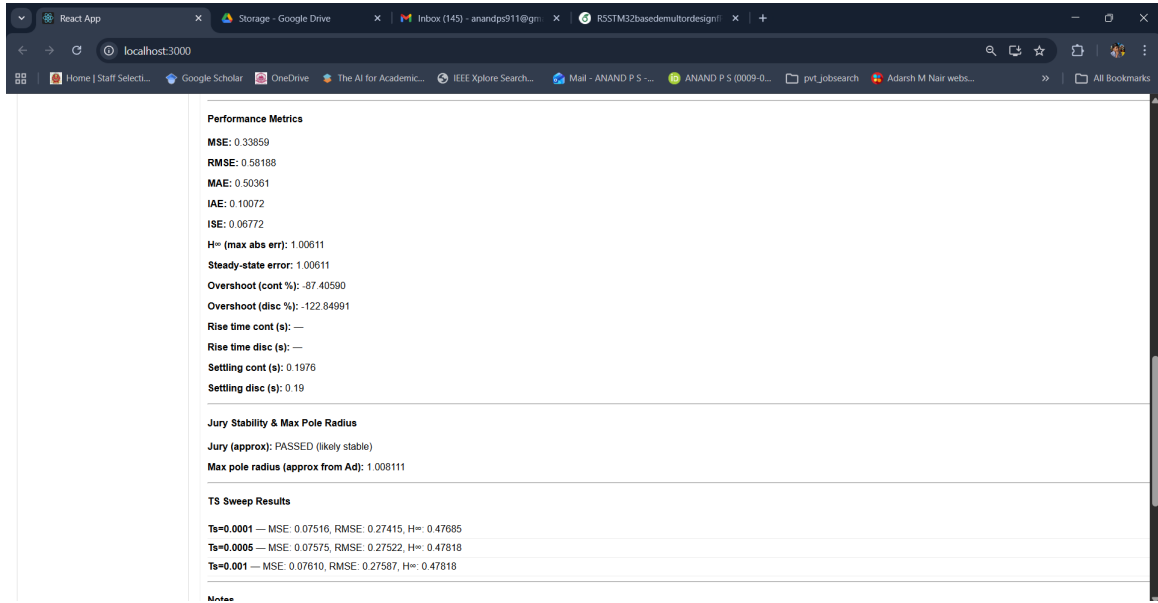


Figure 5.20: Performance Metrics and TS Sweep Results

5.4.9 PMSM Emulator Output

The emulator provides real-time motor output based on input voltage.

For an input of 165 V:

$$\text{Motor Speed} = 3217 \text{ RPM} \quad (5.5)$$

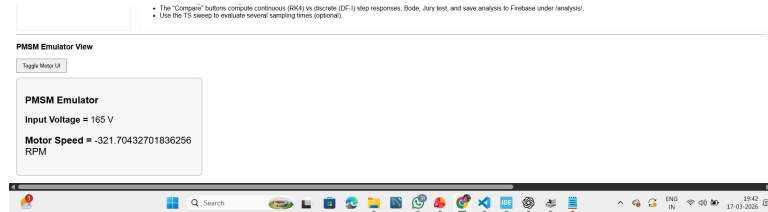


Figure 5.21: PMSM Emulator Output Interface

5.4.10 Validation using Field-Oriented Control (FOC)

To validate the accuracy and dynamic behavior of the developed PMSM emulator, a Field-Oriented Control (FOC) scheme was implemented in MATLAB/Simulink. The emulator was treated as the plant model, and its response was analyzed under closed-loop control.

FOC is a widely used control strategy for PMSM drives, where the stator currents are transformed into a rotating reference frame to independently control torque and flux components. This enables high-performance control with fast dynamic response and improved stability.

FOC System Model

The complete FOC-based PMSM control structure implemented in Simulink is shown in Fig. 5.22.

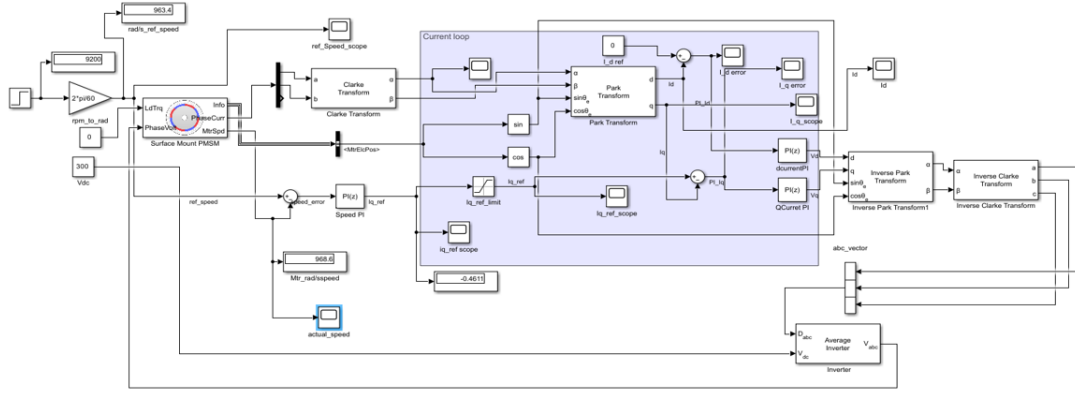


Figure 5.22: FOC-based PMSM control model implemented in MATLAB/Simulink

Validation Results

The performance of the PMSM under FOC control was evaluated by observing the motor speed response and system waveforms. The results are shown in Fig. 5.23.

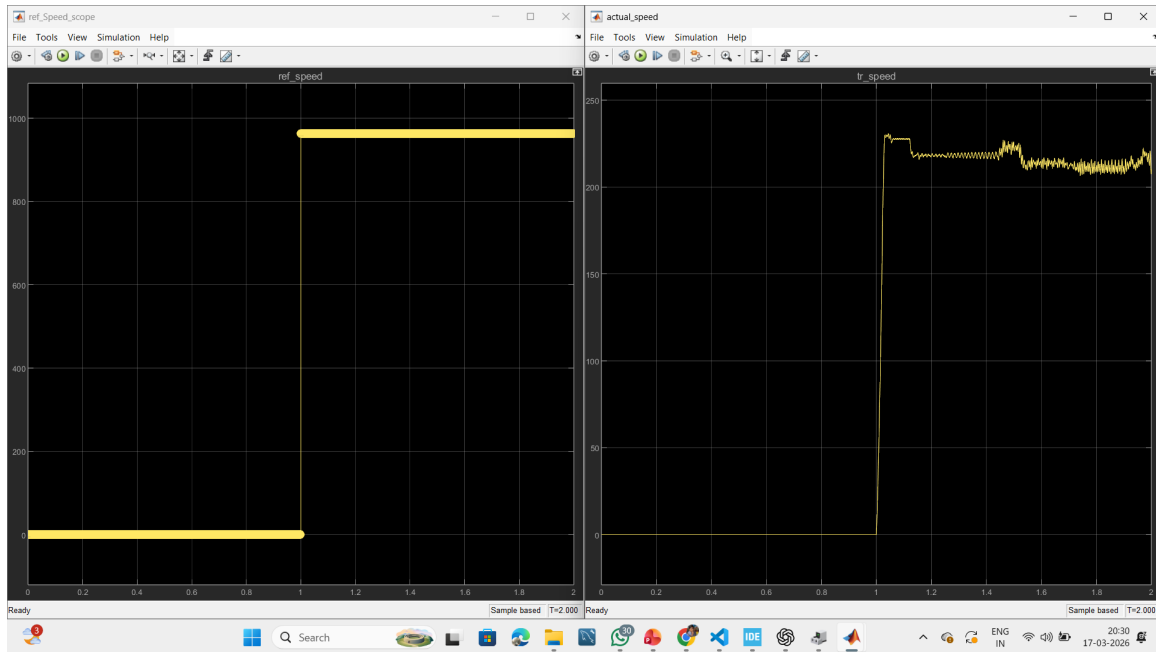


Figure 5.23: FOC validation of PMSM characteristics

The results demonstrate that the emulator successfully tracks the control signals generated by the FOC algorithm and provides a smooth and stable response in terms of speed. The dynamic response of the emulator also closely resembles that of a real PMSM drive, thus validating its effectiveness in simulating PMSM drives using sophisticated control techniques.

Discussion

The ease of integration of the emulator with the FOC controller indicates its readiness for use in a real-time control environment. It helps to save costs and time involved in the development

of control algorithms for the motor.

Thus, the emulator serves as a reliable platform for rapid prototyping and validation of PMSM control strategies in embedded systems.

5.4.11 Conclusion

The developed UI successfully integrates control system analysis with real-time emulation. It provides an intuitive platform for comparing continuous and discrete models while evaluating system performance and stability.

Chapter 6

Conclusion

6.1 Conclusion

This work will present the entire process of developing and validating a Permanent Magnet Synchronous Motor Emulator from scratch. The entire process, including modeling, discretization, and developing a user interface, will be presented, and finally, the deployment of the developed emulator on an embedded system will be shown. The process will start with the derivation of the Continuous-Time Permanent Magnet Synchronous Motor model based on the simulation results obtained from ANSYS Motor-CAD.. The discretization techniques like Zero Order Hold (ZOH), First Order Hold (FOH), Tustin's bilinear transformation, and Matched Pole-Zero mapping are tested and evaluated for a wide range of sampling time intervals. The ZOH technique is proved to be the best among the tested techniques in terms of stability and accuracy. The optimal sampling time interval is determined as 0.01 s, which is the best value that provides the most accurate results without compromising the processing time.

A React-based user interface is designed and developed, which provides the necessary tools and facilities required for the interaction and visualization of the PMSM Emulator results and responses in real-time. The discretized PMSM model is then implemented in an embedded platform like STM32 microcontroller using the Direct Form II technique, which proves the feasibility and effectiveness of the PMSM Emulator.

The PMSM Emulator is further validated in a Field-Oriented Control (FOC) configuration using the MATLAB/Simulink platform, in which the PMSM Emulator is used as the plant in a closed-loop control configuration. The speed response and dynamics are in accordance with the expected PMSM characteristics, thus validating the PMSM Emulator in terms of accuracy and reliability.

In summary, the designed PMSM Emulator is a reliable tool that provides the necessary facilities and benefits in the development and testing of motor control systems in a cost-effective and time-efficient manner.

References

- [1] A. Tashakori, M. Hassanudeen and M. Ektesabi, “FPGA based controller drive of BLDC motor using digital PWM technique,” *2015 IEEE 11th International Conference on Power Electronics and Drive Systems*, pp. 658–662, 2015, doi: 10.1109/PEDS.2015.7203584.
- [2] D. Rama and T. Ananthan, “FPGA Based Emulator Design of a DC Motor,” *2019 International Conference on Communication and Electronics Systems (ICCES)*, pp. 1739–1743, 2019, doi: 10.1109/ICCES45898.2019.9002133.
- [3] V. S. Rosa, V. I. Gervini, S. C. P. Gomes and S. Bampi, “A hardware DC motor emulator,” *2010 First IEEE Latin American Symposium on Circuits and Systems (LASCAS)*, pp. 45–48, 2010, doi: 10.1109/LASCAS.2010.7410216.
- [4] X. Zou, X. Xiao, P. He and Y. Song, “Permanent Magnet Synchronous Machine Emulation Based on Power Hardware-in-The-Loop Simulation,” *2019 IEEE International Electric Machines & Drives Conference (IEMDC)*, pp. 248–253, 2019, doi: 10.1109/IEMDC.2019.8785386.
- [5] M. A. Esparza *et al.*, “Real-time emulator of an induction motor: FPGA-based implementation,” *2012 9th International Conference on Electrical Engineering, Computing Science and Automatic Control (CCE)*, pp. 1–6, 2012, doi: 10.1109/ICEEE.2012.6421152.
- [6] T. Sutikno, N. R. N. Idris, A. Z. Jidin and M. Z. Daud, “FPGA based high precision torque and flux estimator of direct torque control drives,” *2011 IEEE Applied Power Electronics Colloquium (IAPEC)*, pp. 122–127, 2011, doi: 10.1109/IAPEC.2011.5779871.
- [7] X. Ba *et al.*, “A Generalized Per-Phase Equivalent Circuit Model of the PMSM With Predictable Core Loss,” *IEEE/ASME Transactions on Mechatronics*, vol. 28, no. 3, pp. 1512–1521, 2023, doi: 10.1109/TMECH.2022.3227271.
- [8] G. Beylkin, R. D. Lewis and L. Monzón, “On the Design of Highly Accurate and Efficient IIR and FIR Filters,” *IEEE Transactions on Signal Processing*, vol. 60, no. 8, pp. 4045–4054, 2012, doi: 10.1109/TSP.2012.2197397.
- [9] N. Jain, S. Rathore and S. K. Singh, “Designing and Evaluation of the Reduced Order IIR Filter Design for Signal De-Noiseing,” *2021 10th IEEE International Conference on Communication Systems and Network Technologies (CSNT)*, pp. 375–380, 2021, doi: 10.1109/CSNT51715.2021.9509567.
- [10] B. N. Getu, “Digital IIR Filter Design using Bilinear Transformation in MATLAB,” *2020 International Conference on Communications, Computing, Cybersecurity, and Informatics (CCCI)*, pp. 1–6, 2020, doi: 10.1109/CCCI49893.2020.9256625.

- [11] R. Dhaouadi, N. Mohan and L. Norum, “Design and implementation of an extended Kalman filter for the state estimation of a permanent magnet synchronous motor,” *IEEE Transactions on Power Electronics*, vol. 6, no. 3, pp. 491–497, 1991, doi: 10.1109/63.85891.
- [12] I. Horowitz and U. Shaked, “Superiority of transfer function over state-variable methods in linear time-invariant feedback system design,” *IEEE Transactions on Automatic Control*, vol. 20, no. 1, pp. 84–97, 1975, doi: 10.1109/TAC.1975.1100825.
- [13] W. Snelgrove and A. Sedra, “Synthesis and analysis of state-space active filters using intermediate transfer functions,” *IEEE Transactions on Circuits and Systems*, vol. 33, no. 3, pp. 287–301, 1986, doi: 10.1109/TCS.1986.1085910.
- [14] R. Brockett, “Poles, zeros, and feedback: State space interpretation,” *IEEE Transactions on Automatic Control*, vol. 10, no. 2, pp. 129–135, 1965, doi: 10.1109/TAC.1965.1098118.
- [15] P. Stoica and M. Jansson, “MIMO system identification: state-space and subspace approximations versus transfer function and instrumental variables,” *IEEE Transactions on Signal Processing*, vol. 48, no. 11, pp. 3087–3099, 2000, doi: 10.1109/78.875466.
- [16] I. Y. Spitsyn *et al.*, “Mathematical Model and Control System for a Linear DC Motor,” *2021 IV International Conference on Control in Technical Systems (CTS)*, pp. 60–63, 2021, doi: 10.1109/CTS53513.2021.9562846.
- [17] F. Rosu, “Recursive Error Division Kalman Filter for Advanced Driver Assistance Radar Systems,” *2020 13th International Conference on Communications (COMM)*, pp. 31–35, 2020, doi: 10.1109/COMM48946.2020.9142013.
- [18] S. Swain, S. Sahoo, M. C. Tripathy and S. Behera, “Modelling of Inverse Fractional-Order Band-pass and Band-stop Filters using Fractional-Elements,” *2022 IEEE International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, pp. 1–6, 2022, doi: 10.1109/ICDCECE53908.2022.9793241.
- [19] A. W. Morales and S. S. Agili, “Analysis of Analog Sampled S-Parameters Data Using DSP Techniques,” *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–10, 2022, doi: 10.1109/TIM.2022.3142777.
- [20] R. Yadav and M. Gupta, “Design of Fractional Order Differentiators and Integrators Using Indirect Discretization Approach,” *2010 International Conference on Advances in Recent Technologies in Communication and Computing*, pp. 126–130, 2010, doi: 10.1109/ART-Com.2010.67.
- [21] Q. Chen, S. Bu and C. Y. Chung, “Small-Signal Stability Criteria in Power Electronics-Dominated Power Systems: A Comparative Review,” *Journal of Modern Power Systems and Clean Energy*, vol. 12, no. 4, pp. 1003–1018, 2024, doi: 10.35833/MPCE.2023.000526.

- [22] N. Genc and H. Hatas, “Speed control of DC motor using FPGA,” *International Journal on Technical and Physical Problems of Engineering (IJTPE)*, vol. 10, no. 37, pp. 59–64, 2018.
- [23] K. Namboothiripad, Mini, “Analyzing the Impact of Discretization Techniques on Real Time Simulation of DC Servo Motor Using FPGA,” *International Journal of Computing and Digital Systems*, vol. 15, no. 1, pp. 1457–1467, 2024.
- [24] M. B. Dembrani, K. B. Khanchandani, and A. Zurani, “Comparative analysis of various adaptive filter structures using Simulink,” in *Proc. Int. Conf. Intelligent Communication, Control and Devices (ICICCD)*, Singapore, 2016, pp. 847–854.
- [25] M. B. Dembrani, K. B. Khanchandani, and A. Zurani, “Comparative analysis of various adaptive filter structures using Simulink,” in *Proc. Int. Conf. Intelligent Communication, Control and Devices (ICICCD)*, Singapore, 2016, pp. 847–854.